

# Utility and occupancy driven pattern analysis for processing dynamic data streams in damped window control

Taewoong Ryu<sup>1</sup>, Doyoung Kim<sup>1</sup>, Seungwan Park, Seongbin Park, Myungha Cho, Hanju Kim, Junyoung Park, Hyeonmo Kim, Unil Yun<sup>\*</sup>

Department of Computer Engineering, Sejong University, Seoul, Republic of Korea

## ARTICLE INFO

Dataset link: <http://fimi.ua.ac.be/data/>

### Keywords:

Damped window control  
Dynamic data analysis  
High utility occupancy pattern  
Pattern analysis

## ABSTRACT

Data analysis is suitable for data control systems by discovering hidden knowledge that is difficult for humans to perceive from huge and complex data. In various data analysis methods, high utility occupancy pattern analysis considers the utility occupancy of each pattern in the corresponding transaction in addition to the profit and quantity of patterns, which is effective for data control systems, including data science fields. However, recent data holds more insightful knowledge when processing real-time generated data. Previous occupancy-based approaches do not handle the relative significance of the latest data. To overcome the limitation, we introduce a new method for discovering high utility occupancy patterns from dynamic data streams where time-sensitive data consistently occurs. The proposed method assigns relative importance to each pattern by considering the temporal aspect of each transaction. Advanced constructing and restructuring processes are utilized in the proposed method for efficiently controlling data according to the time flow of each pattern in dynamic environments. In the pattern expansion process, a new upper bound adopting the decaying factor is suggested to efficiently reduce unnecessary searches for unpromising patterns. Experimental results demonstrate that the proposed method has superior runtime and scalability performance compared to state-of-the-art methods with comparable memory usage. The ablation study underscores how the proposed components contribute to the overall effectiveness of the proposed method. Additional evaluations indicate that the proposed method analyzes insightful result patterns compared to state-of-the-art methods, and a case study demonstrates its applicability to real-time dynamic data control systems.

## 1. Introduction

Data analysis aims to uncover hidden information from large-scale data, beyond the direct perception of users. To accommodate the diversity of real-world data, various pattern analysis methods have been proposed, including frequent pattern analysis [1], uncertain pattern analysis [2], sequential pattern analysis [3,4], periodic pattern analysis [5], regular pattern analysis [6], temporal fuzzy pattern analysis [7], and erasable pattern analysis [8,9]. It has also been applied to diverse domains, including trajectory analysis in road networks [10], activity recognition in smart home environments [11], and the analysis of medical pathway data in hospital settings [12]. Previous pattern analysis methods that solely focus on the pattern occurrence have limitations in uncovering associations of patterns with key characteristics in

real-world data. Unlike the previous methods, high utility pattern analysis [13,14] is more suitable for processing data in the real world, and several approaches have been proposed to analyze quantitative data by considering item quantity [15]. With this concept, utility-driven pattern analysis has been extended to various concepts, including dynamic data processing [16,17], dynamic [18] and negative [19,20] profit data, top-k concept [21,22], closed patterns [23,24], average utility [25,26], and approximate utility [27]. In addition, due to the advantages of the high utility pattern analysis, it has been utilized in real-world domains, including social networks [28], market analysis [29], and taxi operation sequences [30].

In data control systems, the occupancy of patterns is an important aspect to consider. Although high utility pattern analysis methods analyze insightful patterns based on the quantity and profit, they do not

<sup>\*</sup> Corresponding author.

E-mail addresses: [twryu@sju.ac.kr](mailto:twryu@sju.ac.kr) (T. Ryu), [dyokim@sju.ac.kr](mailto:dyokim@sju.ac.kr) (D. Kim), [swpark@sju.ac.kr](mailto:swpark@sju.ac.kr) (S. Park), [sbpark@sju.ac.kr](mailto:sbpark@sju.ac.kr) (S. Park), [mhcho@sju.ac.kr](mailto:mhcho@sju.ac.kr) (M. Cho), [hjukim@sju.ac.kr](mailto:hjukim@sju.ac.kr) (H. Kim), [jypark@sju.ac.kr](mailto:jypark@sju.ac.kr) (J. Park), [hmokim@sju.ac.kr](mailto:hmokim@sju.ac.kr) (H. Kim), [yunei@sejong.ac.kr](mailto:yunei@sejong.ac.kr) (U. Yun).

<sup>1</sup> First author

<https://doi.org/10.1016/j.knosys.2025.114453>

Received 18 June 2025; Received in revised form 21 August 2025; Accepted 7 September 2025

Available online 8 September 2025

0950-7051/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

account for their relative contribution within each transaction. DOFIA [31] was introduced as the first occupancy concept-based method, which extracts patterns that occupy a high proportion of the transactions as well as patterns whose support is no less than a user-defined threshold. Although the support of a pattern is no less than the threshold, it is determined whether to be extracted as the result through a process of confirming whether the pattern has a high occupancy within the data. Due to the property of the occupancy, applications [32,33] with the occupancy concept have emerged, and related algorithms have also been proposed. Meanwhile, the above studies do not regard the distinct importance of items. Shen et al. suggested a method called OCEAN [34], which combines the concept of occupancy with high utility patterns. Based on the concept, there are online retail data analysis and recommendation control systems with stream analysis services [35].

Suppose a transaction in the retail data is the purchase list of a customer, and an item is a purchased product within the transaction. The utility of an item is expressed as the multiplication of the number of times and its price. The utility occupancy of a pattern for the customer is calculated by dividing the utility of the pattern by the utility of the transaction to which the pattern belongs. Since the pattern is analyzed by comprehensive factors, such as quantity, profit, and occupancy, the information is effectively used to promote a new set of purchases. In the recommendation control system, the approach based on the utility occupancy concept enables the customer to select optimized products that the customer is likely to purchase. The process has the merit that the corresponding user can be presented with a list of similarly configured products as an advertisement. Besides, approaches for uncovering high utility patterns in dynamic circumstances have recently been studied, including real-time detection technology [36] and customer behavior analysis [37].

However, in dynamic environments where continuously accumulated data are generated, the approaches that process the accumulating data have limitations in that they cannot control the importance of time flow in dynamic data. The damped window control [38,39] utilizes the time-decaying value, namely the decaying factor, to give data different importance according to the occurrence of the data. This advanced approach leads to enhanced recommendations that reflect recent trends, whereas previous methods did not consider the temporal importance. Motivated by this, unlike the prior utility occupancy approaches, we introduce a novel algorithm for high utility occupancy pattern analysis utilizing the damped window control, Damped high Utility Occupancy pattern Control and Analysis (DUOCA). DUOCA assigns relative importance to data by considering their arrival times in dynamic environments and efficiently conducting the pattern analysis process in a depth-first search, called DFS, manner with a list-based structure. The proposed method analyzes recent trends in patterns by considering the relative importance of each transaction, as well as historical trends through utility occupancies, by utilizing the decaying factor. To our best knowledge, DUOCA is the first technique for uncovering damped high utility occupancy patterns by considering the temporal aspect. The major contributions and novelties to our study are as follows:

- 1) An efficient approach, DUOCA, is proposed for extracting high utility occupancy patterns in time-sensitive dynamic data by controlling the time flow of each pattern. The DUOCA approach, based on the utility occupancy concept, introduces a novel approach that helps discover temporal-based patterns by giving patterns different importance based on the arrival time of transactions.
- 2) The proposed DUOCA efficiently performs the pattern analysis procedure in a DFS manner by utilizing a list-based structure and upper bound values that handle the damped utility occupancy. In addition, new pruning strategies reflecting the damped window control efficiently reduce the search space in the pattern expansion process without loss or duplication.
- 3) An analysis of comprehensive experiments with multiple types of data is described to obtain a wide variety of evaluation results. Based on the results, it is demonstrated that DUOCA surpasses state-of-the-art comparative methods in the aspects of runtime and scalability with comparable memory usage.
- 4) A case study with the concept-drifting test shows the practical effectiveness of DUOCA, efficiently performing with superior runtime and memory usage performance. Statistical analysis indicates DUOCA analyzes refined results, and the ablation study highlights the impact of proposed strategies.

The remaining contents are as follows: Section 2 describes the studies related to the subject of this paper and their algorithms. Subsequently, in Section 3, the DUOCA approach is explained in depth, in which several examples and figures appear. In addition, a comprehensive description of the algorithms of DUOCA is presented with the analysis of the complexity. In Section 4, the description of various performance evaluations is given, and their results are examined with those of the state-of-the-art approaches to illustrate the efficiency of DUOCA. Finally, the conclusion of this paper is described in Section 5.

## 2. Related work

### 2.1. Occupancy pattern analysis

In traditional pattern analysis methods, which analyze data by considering the frequency of each pattern, previous methods did not address the percentage of each pattern within the entire data. These methods utilize the notion of the support of the pattern without considering the characteristics of the transaction to determine the result patterns. To overcome these limitations, occupancy pattern analysis was proposed to consider the occupancy of each pattern in the data. Occupancy pattern analysis not only considers the support of each pattern, as in frequent pattern analysis, but also considers the relative occupancy of the transaction that belongs to each pattern from the entire data. Accordingly, occupancy pattern analysis is regarded as a pattern analysis that considers the distributions and features of the data. DOFIA [31] is suggested as a method considering the proportion of a pattern in the transaction where it occurs to analyze high utility occupancy patterns. The method extracts promising patterns by comparing the average occupancy of a pattern with a given threshold. DOFRA [40] performs high occupancy pattern analysis on sequential data. HEP [41] proposed an accumulated occupancy concept. The method returns results considering both support and occupancy with one user-defined threshold. HOIMTO [42] designed transaction occupancy as an approach for distinguishing the occupancy of identical patterns in different. While all the above methods process already gathered data in a static manner, HOMI [43] was proposed as a list-based method that analyzes high occupancy patterns in data streams. OCEAN [34] was the first to present the definition of utility occupancy. The utility occupancy of a pattern is designated as the sum of its utility occupancies that the pattern occupies in the corresponding transactions divided by the support of the pattern.

However, Gan et al. [44] proposed a new approach to address inaccurate results and the inefficient traversal of the search space encountered by OCEAN. This list-based approach, called HUOPM, uses four pruning techniques that are not present in OCEAN. Accordingly, HUOPM showed better performance by accomplishing pattern combinations more effectively. HUOMI [45] is suggested as a first-addressed dynamic approach that handles high utility occupancy patterns and maintains the data structures with a single scan. It incorporates a new pruning method to reduce the costs of traversing and sorting processes. Both HUOPM and HUOMI aim to extract high utility occupancy patterns from utility-driven data by utilizing list-based structures. HUOMIL [46] incorporates an indexed-list structure to discover utility occupancy patterns with fewer comparison operations during the pattern expansion but cannot operate in dynamic data environments. IUOIL [47] is also an

**Table 1**

A comparison table with key aspects of methods handling high utility occupancy patterns.

| Approach | Type of data structure | Pruning strategy                   | Use of decaying factor | # of scans | Note                                                       |
|----------|------------------------|------------------------------------|------------------------|------------|------------------------------------------------------------|
| DUOCA    | List-based             | Support, lubduo, ubduo             | Yes                    | 1          | Consider the temporal importance in stream data            |
| IUOIL    | Indexed list-based     | Support, lubuo, ubuo               | No                     | 1          | Process stream data, but does not consider temporal aspect |
| HUOMIL   | Indexed list-based     | Support, lubuo, ubuo               | No                     | 2          | Does not process stream data                               |
| HUOMI    | List-based             | Support, (Suo+Sruo)/[minSup], ubuo | No                     | 1          | Process stream data, but does not consider temporal aspect |
| HUOPM    | List-based             | Support, (Suo+Sruo)/[minSup], ubuo | No                     | 2          | Does not process stream data                               |

**Table 2**

Example of dynamic data streams.

| Database        | TID            | Transaction (Item: Quantity) | TU |
|-----------------|----------------|------------------------------|----|
| DB <sub>0</sub> | T <sub>1</sub> | A : 1, B : 2, C : 2, F : 1   | 18 |
|                 | T <sub>2</sub> | A : 2, B : 1, E : 3          | 16 |
|                 | T <sub>3</sub> | B : 1, E : 2, F : 1, G : 3   | 15 |
|                 | T <sub>4</sub> | C : 3, D : 2, F : 2          | 27 |
| DB <sub>1</sub> | T <sub>5</sub> | B : 3, C : 1, E : 2, G : 2   | 15 |
|                 | T <sub>6</sub> | A : 1, C : 3, F : 2, G : 1   | 26 |
| DB <sub>2</sub> | T <sub>7</sub> | B : 3, D : 2, E : 2          | 13 |
|                 | T <sub>8</sub> | B : 2, C : 1, F : 3, G : 2   | 23 |

**Table 3**

Example of profit table.

| Item   | A | B | C | D | E | F | G |
|--------|---|---|---|---|---|---|---|
| Profit | 3 | 1 | 4 | 2 | 3 | 5 | 1 |

indexed-list-based approach. It extracts results based on the indexed list data structure and handles dynamic environments. Both HUOMIL and IUOIL utilize indexed-based structures to extract result patterns by comparing the TID information of each entry in the process of generating nodes for extended patterns in the pattern expansion process. However, these prior list-based methods have the limitation that they extract high occupancy patterns without considering the importance of data over time. Although several recent methods, which are HUOMI and IUOIL, handle data in stream environments, they have limitations in controlling the latest trends as more important than past data. Therefore, they fail to adequately reflect recent trends. Table 1 introduces the key features of the proposed method and state-of-the-art methods that analyze high utility occupancy patterns, along with the key differences between them.

## 2.2. Pattern analysis based on damped window control

The damped window control is utilized for handling time-sensitive data streams with the temporal aspect of the data. Methods derived

from the damped window control aim to discover patterns considering the recent trends. The decaying factor, an important element in this approach, is configured with a real number between 0 and 1. The factor is multiplied by the utility of a pattern or item belonging to old transactions. It also serves to relatively lower the utility of past transactions by multiplying according to the gap separating the current time and the time at which the corresponding transaction occurred. Consequently, a variety of methods based on this approach have been suggested to consider the relative importance of the latest transactions. DHUPL [39] extracts recent patterns with high utilities using the decaying value and a list-based structure. ILDHUP [48] employs a data structure derived from an indexed-list structure to significantly lower the number of comparisons. In addition to the above works, the damped window control has been used in DSHUP [38] and RHUPS [49], which are also utility-driven methods but consider the sliding window technique when processing data streams. These methods handle the latest transactions to consider the recent trends in the data, but they do not assign relative importance to each pattern. Besides, DMAUP [17] and DEPL [50], which are average utility-based and erasable-based pattern analysis approaches, respectively, have been studied. However, the arrival time of transactions has not been controlled in high utility occupancy patterns. Therefore, a damped window control-based high utility occupancy pattern analysis approach is suggested to control the relative time flow of patterns.

## 3. List-based damped high utility occupancy pattern analysis

### 3.1. Preliminaries

In this subsection, fundamental concepts utilized in the existing utility occupancy methods [34,45] are presented. Table 2 indicates the dynamic data streams employed in the examples described in this paper. Data streams consist of several transactions, and each transaction includes item and quantity pairs. The collection of items within the data is represented by  $I = \{i_1, i_2, \dots, i_n\}$ . Transactions have a distinct identifier,  $tid$ , and  $T_k \subseteq I$  and  $DB = \{T_1, T_2, \dots, T_m\}$  are satisfied for transaction  $T_k$ . The dynamic data streams in Table 2 are composed of one original data stream and two additional data streams. Table 3 exhibits a profit table that represents the external utilities of distinct items. DUOCA utilizes a parameter called the decaying factor because the method is based on the damped window approach technique [17,51]. The decaying value reduces utilities of patterns over time, which is denoted as  $f$ , which is a real number between 0 and 1. A pattern belonging to a transaction  $T_d$  is defined as  $P$ , and the items belonging to  $P$  are defined as  $i_p$ . When the profit and quantity of  $i_p$  are marked as  $eu(i_p)$  and  $iu(i_p, T_d)$ , respectively, the utilities of  $i_p$  and  $P$  in  $T_d$  are defined as follows:  $u(i_p, T_d) = eu(i_p) \times iu(i_p, T_d)$  and  $u(P, T_d) = \sum u(i_p, T_d)$ , where  $i_p \in P \subseteq T_d$ . The transaction utility of  $T_d$  is expressed as  $tu(T_d)$  and is computed as follows:  $tu(T_d) = \sum u(i_p, T_d)$ , where  $i_p \in T_d$ . The support of  $P$  is calculated as the number of transactions where  $P$  exists and is denoted as  $sup(P)$ . The utility occupancies of pattern  $P$  in  $T_d$  and  $DB$  are represented as  $uo(P, T_d)$  and  $uo(P)$ , respectively. These values are computed as follows:  $uo(P, T_d) = u(P, T_d)/tu(T_d)$  and  $uo(P) = \sum uo(P, T_d)/sup(P)$ , where  $P \subseteq T_d \subseteq DB$ . For arbitrary items  $i_p$  and  $i_q$ , if  $i_p$  precedes  $i_q$  in the global list structure, it is denoted as  $i_p < i_q$ . The remaining utility occupancy of pattern  $P$  in  $T_d$  and  $DB$  are represented as  $ruo(P, T_d)$  and  $ruo(P)$ , respectively. If the last item of pattern  $P$  is  $i_p$  and items located after  $i_p$  are  $i_q$ ,  $ruo(P, T_d)$  and  $ruo(P)$  are calculated as follows:  $ruo(P, T_d) = \sum uo(i_y, T_d)$  and  $ruo(P) = \sum ruo(P, T_d)/sup(P)$ , where  $i_x < i_y$  and  $P \subseteq T_d \subseteq DB$ . If the utility occupancy and the support of a pattern are no less than the minimum utility occupancy threshold,  $\beta$ , and the minimum support threshold multiplied by the number of scanned transactions,  $\alpha \times |DB|$ , respectively, the pattern is regarded as a high utility occupancy pattern.

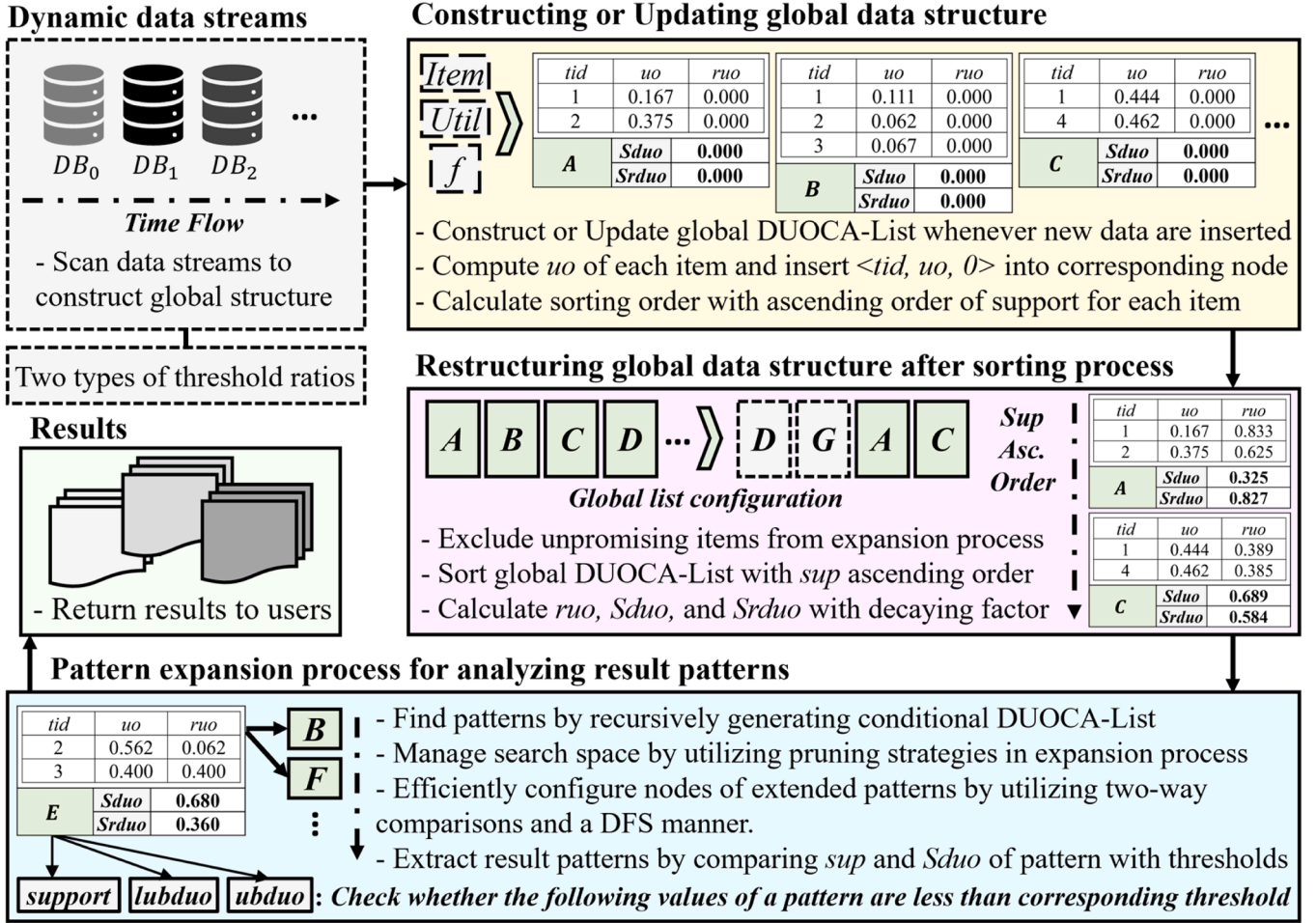


Fig. 1. Overall architecture of DUOCA.

### 3.2. Overall procedure of discovering high utility occupancy pattern with damped window

Fig. 1 represents the whole flow of the proposed method. The method comprises three procedures: 1) *Construction Procedure*, 2) *Restructure Procedure*, and 3) *Pattern Expansion Procedure*. In the *Construction Procedure*, the algorithm addresses the transactions of the scanned data and builds the global DUOCA-List. This process is repeated whenever new data are scanned, and the global list is renewed with additional information. After that, the *Restructure Procedure* is executed. In this procedure, the global list is sorted in the ascending order of the support and calculates values called *ruo*, *Sduo*, and *Srduo* (illustrated in detail in Section 3.4). In this case, the *ruo* calculation is carried out, and the calculation of *Sduo* and *Srduo* is performed after the corresponding decaying value is calculated. Afterward, the *Pattern Expansion Procedure* runs with the restructured global list, which aims to discover damped high utility occupancy patterns while generating the conditional lists with two-way comparisons. In the third procedure, the pattern expansion process is conducted in a DFS manner and recursively extracts patterns of all lengths. When the procedure is finished, the results are returned to users, and the method waits for the insertion of additional data.

### 3.3. Constructing global data structure with original data

This section describes the DUOCA-List devised to manage damped occupancy information of a pattern and transactions to which the pattern belongs. It consists of a global DUOCA-List, and the global list is

involved in finding result patterns in the *Pattern Expansion Procedure*. DUOCA-List does not generate candidate patterns since it stores and holds all information necessary for pattern extraction. Therefore, the proposed approach discovers results efficiently without extra data scans.

**Definition 1.** (Damped utility occupancy of a pattern): The damped utility occupancy of pattern  $P$  in transaction  $T_d$  is notated as  $duo(P, T_d)$ , and it is a value that multiplies the decaying factor to the utility occupancy of  $P$  in  $T_d$ . When the final  $tid$  in the data is  $T$ , the  $tid$  of  $T_d$  is  $t$ , and the decaying factor is  $f$ ,  $duo(P, T_d)$  is computed as follows:

$$duo(P, T_d) = \frac{u(P, T_d)}{tu(T_d)} \times f^{(T-t)} \quad (1)$$

The damped utility occupancy of  $P$  in data  $D$  is computed by aggregating the  $duo$  of all transactions involving  $P$ , then dividing this aggregate by  $Sup(P)$ . It is represented by the following formula:

$$duo(P) = \frac{\sum_{P \subseteq T_d \in D} duo(P, T_d)}{Sup(P)} \quad (2)$$

**Definition 2.** (Remaining damped utility occupancy of a pattern): The damped remaining utility occupancy of pattern  $P$  in transaction  $T_d$  is denoted as  $rduo(P, T_d)$ , which is a value that multiplies the decaying factor,  $f$ , by the remaining utility occupancy of  $P$  in  $T_d$ . When the last  $tid$  in the data is  $T$ , the  $tid$  of  $T_d$  is  $t$ ,  $rduo(P, T_d)$  is calculated as follows:

$$rduo(P, T_d) = \frac{\sum u(i_y, T_d)}{tu(T_d)} \times f^{(T-t)}, \text{ where } i_y \succ P \wedge i_y \in T_d \quad (3)$$



The remaining damped utility occupancy of  $P$  in  $DB$  is computed by accumulating the  $ruo$  of all transactions involving  $P$ , then dividing this accumulated value by  $Sup(P)$ . It is represented as  $rduo(P)$  and is computed as follows:

$$rduo(P) = \frac{\sum_{P \subseteq T_d \in DB} rduo(P, T_d)}{Sup(P)} \quad (4)$$

**Definition 3.** (Damped high utility occupancy pattern): For a given data,  $DB$ , the minimum support  $minSup(DB)$  is obtained as  $|DB| \times \alpha$ , where  $\alpha$  ( $0 < \alpha \leq 1$ ) is a user-defined threshold. Also, the minimum utility occupancy is declared as  $\beta$  ( $0 < \beta \leq 1$ ), which is another threshold given by the user. Pattern  $P$  is a damped high utility occupancy pattern, if  $Sup(P)$  is no less than  $minSup(D)$  and  $duo(P)$  is no less than  $\beta$ .

The components and construction process of DUOCA-List are introduced below. Assume that the DUOCA-List contains multiple nodes and each node stores various information about a pattern. To compactly keep the information of each pattern occurring per different transaction, each node consists of the following elements: name of the pattern, a set of entries,  $Sduo$ , and  $Srduo$ . An entry stores the information of a transaction and consists of  $tid$ ,  $uo$ , and  $ruo$ . Two types of DUOCA-Lists are created during the entire process. One is a global DUOCA-List. It stores information gathered in previous incremental steps and is maintained without any deletion during the entire process. The entries in each node of the global list are not modified by the decaying factor to process the damped approach efficiently, and the details are illustrated in the following section. The other is a conditional DUOCA-List. It stores the values of extended patterns identical to the global DUOCA-List. When the construction process starts, the proposed DUOCA scans transactions in the data. The utility of each transaction is computed to calculate the utility occupancy of each pattern. The items appearing in the transaction are scanned in sequence. If the node in accordance with an item does not exist in the global list, the corresponding node of the item is newly generated. Then,  $Sduo$  and  $Srduo$  are initialized to 0. A new entry is incorporated into the node where the  $ruo$  of the entry is initialized to 0. The utility occupancy, which is the utility of the item divided by the transaction utility, is stored in  $uo$ . The calculation of  $uo$ ,  $Sduo$ , and  $Srduo$ , where the decaying factor is processed, is conducted in the restructuring phase.

**Example 1.** This example explains the construction procedure of the global DUOCA-List from the original data. The utility of item  $C$  is calculated as  $2 \times 4 = 8$ , and  $tu(T_1)$  is the sum of utilities of items  $A$ ,  $B$ ,  $C$ , and  $F$ , which is 18. Afterward, the node generation of item  $A$  proceeds. Since the node corresponding to  $A$  does not exist in the global list, a new node of item  $A$  is generated. The  $Sduo$  and  $Srduo$  of the node are initialized to 0, and an entry corresponding to  $T_1$  is inserted into the node of item  $A$ . The  $ruo$  of the entry is initially established to 0, the utility of item  $A$  is divided by  $tu(T_1)$ , and the divided value is stored in  $uo$ . The node generations of items  $B$ ,  $C$ , and  $F$  are carried out in the identical way as the node of item  $A$ .

**Lemma 1.** In the damped window control system, items appearing in the latest transactions have relatively higher importance than the items appearing in previously occurring transactions.

**Proof.** Assume that pattern  $X$  appears in  $T_1, T_2, \dots, T_k$  and pattern  $Y$  occurs at  $T_{k+1}, T_{k+2}, \dots, T_{2k}$ , where both patterns have the same utility occupancy set as  $uo(X, T_1) = uo(Y, T_{k+1}), uo(X, T_2) = uo(Y, T_{k+2}), \dots, uo(X, T_k) = uo(Y, T_{2k})$ . When  $T_{2k}$  is the last transaction, the damped utility occupancy is calculated as follows:

$$duo(X) = uo(X, T_1) \times f^{(2k-1-k)} + uo(X, T_2) \times f^{(2k-2-k)} + \dots + uo(X, T_k) \times f^{(2k-k)} \quad (5)$$

$$duo(Y) = uo(Y, T_1) \times f^{(2k-1-k)} + uo(Y, T_2) \times f^{(2k-2-k)} + \dots + uo(Y, T_k) \times f^{(2k-k)} \quad (6)$$

According to the equations, the following relationships are satisfied.

$$duo(X) = duo(Y) \times f^k \quad (7)$$

$$duo(Y) = duo(X) / f^k \quad (8)$$

Since,  $f$  is always less than 1,  $f^k$  is also less than 1. Therefore,  $duo(Y)$  is greater than  $duo(X)$ , even though  $X$  and  $Y$  have the same utility occupancy set.

#### 3.4. Restructuring process of constructed global DUOCA-List

In this section, an improved restructuring process is explained for efficiently extracting damped high utility occupancy patterns. When a request to find results by a user arrives, realigned conditional lists are structured using the global list. In stream environments, all information, including previous data, is managed in the global list. That is, even if there are nodes for unpromising items among the nodes generated by the current information, additional data scans allow nodes for unpromising items to be considered promising items. In addition, the amount of the decaying factor multiplied by a transaction is also constantly modified. Therefore, the tasks of realignment, pruning, and the decaying factor application are achieved together in the restructuring process to an efficient result pattern discovery. When the restructuring procedure is launched by a request to find results, the global list is sorted in the support ascending order of distinct items. Since the support of each item satisfies the anti-monotone property, items whose utilities are less than  $minSup$  are pruned. In this process, the relative importance compared to the latest transaction is applied to the  $uo$  of each entry for calculating the  $duo$ . Whenever the  $duo$  is calculated for each entry in a node, it is accumulated in  $Sduo$  of the corresponding node. After the traversal of all entries is completed, the  $Sduo$  is newly updated by dividing the current  $Sduo$  value by the number of entries. Moreover, the  $ruo$  values of entries in each node are also computed since the global list is sorted according to the support ascending order. In the  $ruo$  calculation phase, a temporary array called *temp-Table* is created. The *temp-Table* comprises *Suo* and *tid*. The *Suo* stores the sum of  $uo$  for visited nodes, and the *tid* is used to identify corresponding transactions. The entries of the table are generated according to the transaction number of the scanned data, which enables the efficient calculation of  $ruo$  for each entry. For efficient  $ruo$  computation, the conditional list is navigated in reverse of the sorting order. The *tid* of the current list is analyzed against the *tid* in the *temp-Table*. If both *tid* values are the same, the *Suo* of the *temp-Table* is duplicated to  $ruo$  of the entry. Next, the  $duo$  of the entry is added to the *Suo* of the corresponding entry in the *temp-Table*. The remaining traversal proceeds in an identical way, and the restructured global list is returned when the traversal is done.

**Example 2.** This example introduces the process of calculating  $Sduo$  and  $Srduo$  of the global list in the restructuring process. The proposed method utilizes the decaying factor after sorting the global list to calculate the  $Sduo$  and  $Srduo$ . Suppose that the minimum support threshold is configured to 0.3. Since the size of  $DB_0$  is 4,  $minSup$  is calculated as  $4 \times 0.3 = 1.2$ . Since the support values of items  $D$  and  $G$  are less than 1.2, these items are pruned. In the global list, the first entry for item  $A$  has the  $uo$  of 0.167. When the decaying factor is set to 0.8, entries with *tid* 1 are multiplied by  $0.8^{4-1}$ , and the  $duo$  of item  $A$  in  $T_1$  is calculated as  $0.167 \times 0.8^{4-1} = 0.086$ . Accordingly, 0.086 is added to the  $Sduo$  of item  $A$ . The *temp-Table* is generated based on information of item  $F$  located at the last position of the global list according to the reverse of the sorting order. Entries {1, 3, 4} and the corresponding  $duo$  values are inserted into the *temp-Table*. The remaining nodes of the

|                   |              |              |
|-------------------|--------------|--------------|
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 4                 | 0.154        | 0.000        |
| <b>D</b>          | <i>Sduo</i>  | <b>0.000</b> |
|                   | <i>Srduo</i> | <b>0.000</b> |
| Pruned by support |              |              |
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 2                 | 0.562        | 0.062        |
| 3                 | 0.400        | 0.400        |
| <b>E</b>          | <i>Sduo</i>  | <b>0.680</b> |
|                   | <i>Srduo</i> | <b>0.360</b> |
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 1                 | 0.111        | 0.278        |
| 2                 | 0.062        | 0.000        |
| 3                 | 0.067        | 0.333        |
| <b>B</b>          | <i>Sduo</i>  | <b>0.150</b> |
|                   | <i>Srduo</i> | <b>0.409</b> |
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 1                 | 0.167        | 0.833        |
| 2                 | 0.375        | 0.625        |
| <b>A</b>          | <i>Sduo</i>  | <b>0.325</b> |
|                   | <i>Srduo</i> | <b>0.827</b> |
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 1                 | 0.278        | 0.000        |
| 3                 | 0.333        | 0.000        |
| 4                 | 0.385        | 0.000        |
| <b>F</b>          | <i>Sduo</i>  | <b>0.794</b> |
|                   | <i>Srduo</i> | <b>0.000</b> |
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 1                 | 0.444        | 0.389        |
| 4                 | 0.462        | 0.385        |
| <b>C</b>          | <i>Sduo</i>  | <b>0.689</b> |
|                   | <i>Srduo</i> | <b>0.584</b> |

Fig. 2. Reconstructed conditional DUOCA-list.

|                   |              |              |
|-------------------|--------------|--------------|
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 4                 | 0.154        | 0.000        |
| 7                 | 0.308        | 0.000        |
| <b>D</b>          | <i>Sduo</i>  | <b>0.000</b> |
|                   | <i>Srduo</i> | <b>0.000</b> |
| Pruned by support |              |              |
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 1                 | 0.167        | 0.833        |
| 2                 | 0.375        | 0.625        |
| 6                 | 0.115        | 0.885        |
| <b>A</b>          | <i>Sduo</i>  | <b>0.207</b> |
|                   | <i>Srduo</i> | <b>0.905</b> |
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 2                 | 0.562        | 0.062        |
| 3                 | 0.400        | 0.600        |
| 5                 | 0.400        | 0.600        |
| 7                 | 0.462        | 0.231        |
| <b>E</b>          | <i>Sduo</i>  | <b>0.853</b> |
|                   | <i>Srduo</i> | <b>0.705</b> |
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 3                 | 0.200        | 0.400        |
| 5                 | 0.133        | 0.467        |
| 6                 | 0.038        | 0.846        |
| 8                 | 0.087        | 0.913        |
| <b>G</b>          | <i>Sduo</i>  | <b>0.245</b> |
|                   | <i>Srduo</i> | <b>1.825</b> |
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 1                 | 0.444        | 0.389        |
| 4                 | 0.462        | 0.385        |
| 5                 | 0.267        | 0.200        |
| 6                 | 0.462        | 0.385        |
| 8                 | 0.174        | 0.739        |
| <b>C</b>          | <i>Sduo</i>  | <b>0.888</b> |
|                   | <i>Srduo</i> | <b>1.327</b> |
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 1                 | 0.278        | 0.111        |
| 3                 | 0.333        | 0.067        |
| 4                 | 0.385        | 0.000        |
| 6                 | 0.385        | 0.000        |
| 8                 | 0.652        | 0.087        |
| <b>F</b>          | <i>Sduo</i>  | <b>1.223</b> |
|                   | <i>Srduo</i> | <b>0.132</b> |
| <i>tid</i>        | <i>uo</i>    | <i>ruo</i>   |
| 1                 | 0.111        | 0.000        |
| 2                 | 0.062        | 0.000        |
| 3                 | 0.067        | 0.000        |
| 5                 | 0.200        | 0.000        |
| 7                 | 0.231        | 0.000        |
| 8                 | 0.087        | 0.000        |
| <b>B</b>          | <i>Sduo</i>  | <b>0.436</b> |
|                   | <i>Srduo</i> | <b>0.000</b> |

Fig. 3. Reconstructed global DUOCA-List with  $DB_1$  and  $DB_2$ .

global list are restructured by traversing all nodes that are not pruned by the support threshold. Fig. 2 shows the conditional list reconstructed through the above processes.

### 3.5. Update of global list with dynamic data

The process of inserting additional data in stream environments is described in this section. Previous information, including the original data, is not deleted and managed in the global list. Therefore, even if new information is inserted, the data of existing information in the global list is not read again. As additional data is entered, the latest transaction changes and the decaying value are recalculated. If the damped utility occupancy reflecting the decaying factor is managed for each entry of a node, additional data scans cause inefficiency, as the damped utility occupancy of each entry needs to be updated each time. Therefore, the global list stores utility occupancy values to which the decaying factor is not applied. When a transaction is newly scanned, the transaction utility is computed first, and then the information of items in the transaction is processed. If the corresponding node of an item exists in the global list, a new entry is added to the node. Conversely, if there is no corresponding list in the global list, a new list and entry are created. In this case, the construction process in Section 3.3 is followed. The *Sduo* and *Srduo* of the list are initialized to 0. The *uo* of an entry maintains the utility of the item divided by the corresponding transaction utility. Each *ruo* is initialized to 0 since the sorting order can change as transactions are inserted. For the efficiency of the method, the sorting process of the global list is performed upon the request to analyze result patterns. Similarly, the *ruo* calculation is performed in the restructuring process.

**Example 3.** This example illustrates the configuration of the updated global list structure after additional data scans occur. Fig. 3 represents the updated and restructured global DUOCA-List with  $DB_1$  and  $DB_2$ . The global DUOCA-List is arranged in the newly calculated support ascending order, which is denoted as  $D \prec A \prec E \prec G \prec C \prec F \prec B$ . As in the previous instance, the minimum support threshold is set to 0.3. Since the number of scanned transactions increases to 8,  $minSup$  is set to 2.4. Accordingly, item *D* is pruned since the support of item *D* is less than 2.4. Since the first entry in the node of item *A* has a *tid* of 1, the decaying value is calculated as  $0.8^{8-1}$ . The *duo* of item *A* for  $T_1$ , which is computed as  $0.167 \times 0.8^{8-1}$ , is added to the *Sduo* value. In the same way, the corresponding decaying values are assigned to all remaining entries. After that, the *ruo* and *Srduo* values of all nodes are calculated with the *temp-Table*. Then, the restructured global DUOCA-List is returned for conducting the patterns expansion process.

### 3.6. Analyzing high utility occupancy patterns with damped window control

This section introduces the pattern extension process and pruning techniques. The proposed algorithm iterates the restructured global list arranged in the support ascending order and conducts the pattern expansion process. The support of a pattern fulfills the anti-monotone property, which is computed as the number of entries for the corresponding node. The patterns that appear at the front of the obtained global list are integrated with remaining patterns in the global list. If the expansion proceeds from patterns with low support, the pattern expansion can be conducted efficiently. There are the minimum support

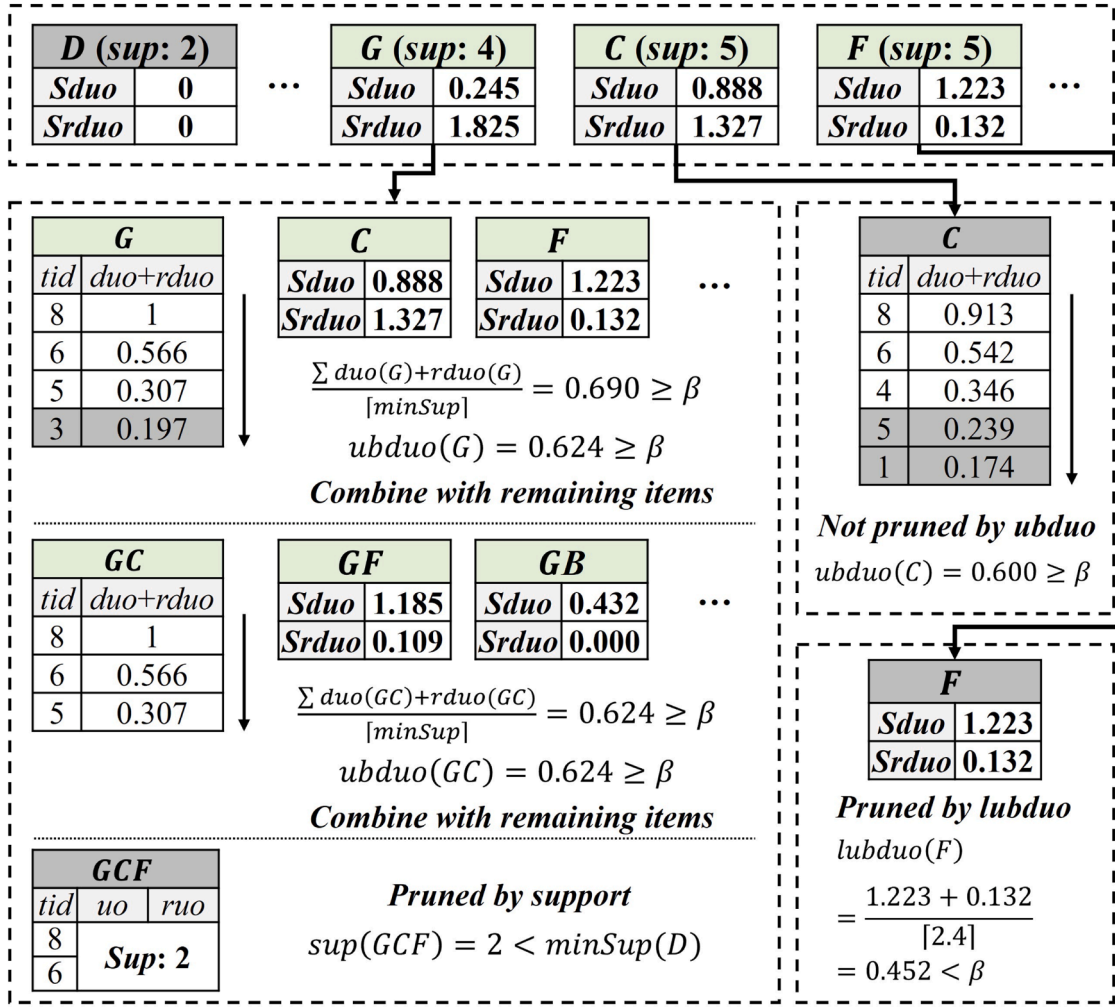


Fig. 4. Combining patterns with pruning strategies.

and utility occupancy thresholds in high utility occupancy pattern analysis. However, the measure of utility occupancy does not fulfill the anti-monotone property. Therefore, pruning methods using the estimated upper bound are utilized in the proposed method.

**Definition 4.** (Pre-pruning upper bound damped utility occupancy): The pre-pruning upper bound for the damped utility occupancy of pattern  $P$  is expressed as  $lubduo(P)$ . This represents a loose upper bound to allow for efficient computation.

$$lubduo(P) = \frac{\sum_{T_d \in DB \wedge P \subseteq T_d} \{duo(P, T_d) + rduo(P, T_d)\}}{minSup} \quad (9a)$$

**Definition 5.** (Upper bound damped utility occupancy): The upper bound for the damped utility occupancy of pattern  $P$  is expressed as  $ubduo(P)$ . Since this is the maximum utility occupancy value that  $P$  can have, it can be employed as a pruning indicator. When  $top[minSup]$  implies selecting the largest  $[minSup]$  number of  $duo+rduo$ ,  $ubduo(P)$  can be estimated as follows:

$$ubduo(P) = \frac{\sum_{topminSup \wedge P \subseteq T_d} \{duo(P, T_d) + rduo(P, T_d)\}}{minSup} \quad (9b)$$

**Lemma 2.** The upper bound damped utility occupancy utilized in the proposed method satisfies the property of anti-monotone.

**Proof.** Suppose that there are patterns  $X$  and  $Y$ , which satisfy the following conditions:  $X \subseteq Y$ . In the ordered transactions,  $duo(Y -$

$X) \leq rduo(X)$ , where  $Y - X$  means the set of items after  $X$ , is always satisfied. This is because the remaining utility occupancy is the sum of utility occupancies of all items that appear after the pattern  $X$ . Accordingly,  $duo(Y)$  is computed as follows:

$$\begin{aligned} duo(Y) &= \frac{\sum_{Y \subseteq T_d \in D} duo(Y, T_d)}{Sup(Y)} = \frac{\sum_{Y \subseteq T_d \in D} \{duo(X, T_d) + duo(Y - X, T_d)\}}{Sup(Y)} \\ &\leq \frac{\sum_{Y \subseteq T_d \in D} \{duo(X, T_d) + rduo(X, T_d)\}}{Sup(Y)} \end{aligned} \quad (10)$$

According to Definition 3, if pattern  $Y$  is a promising pattern,  $sup(Y)$  is always no less than the  $minSup$ . When  $top$  means selecting the greatest  $[minSup]$  number of values in all  $duo+rduo$  values, the above expression is expressed as follows.

$$\begin{aligned} duo(Y) &\leq \frac{\sum_{Y \subseteq T_d \in D} \{duo(X, T_d) + rduo(X, T_d)\}}{Sup(Y)} \\ &\leq \frac{\sum_{topminSup \wedge Y \subseteq T_d} \{duo(X, T_d) + rduo(X, T_d)\}}{minSup} = ubduo(X) \end{aligned} \quad (11)$$

As a result, even if the decaying factor is considered, the upper bound damped utility occupancy satisfies the anti-monotone property.

In the proposed DUOCA, the pruning strategy utilizing  $ubduo$  provides the efficiency of the pattern expansion process. However, the sorting and traversing of information of entries in a node are required to calculate the  $ubduo$  of each pattern. Therefore, a fast pre-running is performed using  $Sduo+Srduo$  before calculating the  $ubduo$ . For intuitive

**Algorithm 1**  
 DUOCA.

---

**Input:** A dynamic data streams,  $DB = \{db_0, db_1, \dots\}$ , a decaying factor  $f$ , the support and utility occupancy thresholds,  $\alpha$  and  $\beta$   
**Output:** A set of damped high utility occupancy patterns,  $HUOP$   
**Variable:** A set of global and conditional DUOCA-lists,  $GL$  and  $CL$ , a last inserted transaction id,  $Ltid$ , the number of transactions in the scanned data,  $dbSize$ , a minimum support threshold,  $minSup$  (global variable)

01. Initialize  $dbSize$  to 0
02. for each database,  $db$ , in  $DB$  do
03.   for each transaction,  $T_k$ , in  $db$  do
04.     Construct  $DUOCA$ -Lists with  $T_k$
05.     Assign the last TID into  $Ltid$
06.    $dbSize \leftarrow dbSize + |db|$
07.    $minSup \leftarrow \alpha \times dbSize$
08.   if there is a request to find results then
09.      $Restructure(GL, Ltid)$
10.      $HUOP \leftarrow Expansion(GL, \emptyset)$
11.   return  $HUOP$  to the user

---

**Algorithm 2**  
 Restructure.

---

**Input:** The global DUOCA-list,  $GL$ , the last inserted transaction id,  $Ltid$   
**Output:** The updated global DUOCA-list,  $GL$   
**Variable:** A decaying value,  $df$

01. Sort  $GL$  in the support ascending order
02. for each node,  $G_i$ , in  $GL$ , do // Reverse of the sorting order
03.   for each entry,  $e$ , in  $G_i$ , do
04.     Calculate  $e.ruo$  traversing the entries in the node
05.      $df \leftarrow pow(f, Ltid - e.tid)$
06.      $G_i.Sduo \leftarrow G_i.Sduo + (e.uo \times df)$  //  $duo$  of each entry is added to  $Sduo$
07.      $G_i.Srduo \leftarrow G_i.Srduo + (e.ruo \times df)$  //  $rduo$  of each entry is added to  $Srduo$
08. return  $GL$

---

**Algorithm 3**  
 Expansion.

---

**Input:** A DUOCA-list for current pattern,  $CL$ , a prefix pattern,  $pref$   
**Output:** A set of damped high utility occupancy patterns,  $HUOP$   
**Variable:** A conditional DUOCA-list for the next level,  $NCL$

01.  $HUOP \leftarrow \emptyset$
02. for each,  $C_i$ , in  $CL$  do // From first  $C_i$  to last  $C_i$
03.   if  $(C_i.sup < minSup)$  then
04.     Continue
05.   if  $(C_i.Sduo / C_i.sup \geq \beta)$  then
06.      $HUOP \leftarrow (pref \cup \{C_i.item\})$
07.   if  $((C_i.Sduo + C_i.Srduo) / [minSup] \geq \beta$  and  $Upperbound(C_i) \geq \beta)$  then
08.      $NCL.clear()$
09.     for each,  $C_j$ , in  $CL$  do //  $i < j$
10.        $C \leftarrow Construct(C_i, C_j, [pref] + 2)$
11.       if  $(C \neq \emptyset)$  then
12.          $NCL.insert(C)$
13.     if  $(|NCL| > 1)$  then
14.        $HUOP \leftarrow HUOP \cup Expansion(NCL, pref \cup \{C_i.item\})$
15. return  $HUOP$

---

understanding, the sum of utility occupancies corresponding to each transaction is stored in  $Sduo$  and  $Srduo$ . However, the average of the utility occupancies calculated for each transaction is not calculated in the former processes. Therefore, when using the pruning strategy utilizing  $Sduo + Srduo$ , the value of  $lubduo$  pruning obtained by dividing  $Sduo + Srduo$  by  $[minSup]$  is compared with the minimum utility occupancy threshold to determine whether to prune the pattern. The  $Sduo + Srduo$  divided by  $[minSup]$  is no greater than  $ubduo$ , since the pruning strategy with  $ubduo$  utilizes the value dividing the sum of the greatest  $[minSup]$  number of values in all  $duo + rduo$  values at each node by  $[minSup]$ . Therefore, if the upper bound using  $Sduo$  and  $Srduo$  is less than  $\beta$ , which is a user-defined minimum utility occupancy threshold, the pattern is pruned without the  $ubduo$  calculation. The pattern

expansion process proceeds in a DFS manner and two-way comparisons for efficiently configuring entries in a node, and the pattern length is extended by recursively performing the pattern expansion process. First, the pruning strategy using the support is conducted. If the support of a pattern is less than  $minSup$ , the method terminates the extension of the item and shifts to the next pattern according to the sorting order. Otherwise, the check for the pruning strategy using  $lubduo$  is performed. Separately, by comparing the  $duo$  of the current pattern to  $\beta$ , it is concluded whether the pattern is a result pattern. When the upper bound using  $Sduo$  and  $Srduo$  is less than  $\beta$ , the expansion is terminated. Otherwise, the pruning strategy utilizing the  $ubduo$  is performed. If the  $ubduo$  of a pattern also exceeds  $\beta$ , the node of the pattern is integrated with the node of the following other patterns to generate a conditional list for a super pattern. The entries in each of the previous two nodes are compared to generate a new node in the conditional list. When both entries in each node have a common  $tid$ , a new entry is generated and embedded in the new node. The  $uo$  of the new entry is computed by combining the  $uo$  of the previous two nodes and excluding the  $uo$  of the common prefix. Also, the  $ruo$  of the new entry is set as the smaller value. For this reason, the  $ruo$  of the item that appears later is selected. While entries are generated, the  $duo$  and  $ruo$  are computed according to the newly generated  $uo$  and  $ruo$ , and the  $duo$  and  $ruo$  are reflected in  $Sduo$  and  $Srduo$ . Regarding the pattern expansion process, the nodes for length- $k$  patterns generate nodes for length- $(k+1)$  patterns until there are no further lists to combine. In this way, all nodes are traversed, and damped high utility occupancy patterns are extracted without pattern loss.

**Example 4.** This example describes the pattern expansion process utilizing items  $G$  and  $C$ . Fig. 4 represents combination examples of several patterns with pruning strategies. The minimum utility occupancy,  $\beta$ , is configured at 0.6. Since the support of item  $G$  is 4, item  $G$  is not pruned. However, it is not included in the result set because its  $Sduo$  is less than  $\beta$ . Even if the pattern is invalid, the  $lubduo(G)$ ,  $(Sduo + Srduo) / [minSup]$  value of item  $G$  is calculated as  $2.07 / [2.4] = 0.69$ , which is no less than  $\beta$ . Since  $ubduo(G)$ , which is 0.624, is also no less than  $\beta$ , it is combined with the next item  $C$  to create a new node. The node corresponding to the pattern  $\{GC\}$  is appended to the conditional list of length-2 patterns. In the same way, the remaining nodes for the prefix item  $G$  are constructed. Next, the algorithm combines the nodes of length-2 patterns to create the conditional list of length-3 patterns by combining the two length-2 patterns, patterns  $\{GC\}$  and  $\{GF\}$ . The procedure is the same as before. Pattern  $\{GC\}$  is also not pruned since  $ubduo(GC)$  is no less than  $\beta$ . Item  $C$  is also not pruned by  $ubduo$  because  $ubduo(C)$ , which is 0.600, is no less than  $\beta$ . On the contrary, item  $F$  is pruned by the  $lubduo$  pruning strategy since  $(1.223 + 0.132) / [2.4] = 0.452$ , which is less than  $\beta$ .

### 3.7. Algorithm for dynamic analysis of high utility occupancy patterns with damped control

In this section, detailed operations of the proposed algorithm are explained. In Algorithm 1, the dynamic data streams, decaying factors, and thresholds are required as input values. To compute  $minSup$ , the number of transactions in the scanned data is measured. Thus,  $dbSize$ , which means the scale of data, is initialized to 0 (Line 01). The original and additional data are read sequentially (Lines 02–11). The scanned data is processed separately for each transaction (Lines 03–05). According to the transactions, the global lists are constructed (Line 04). To calculate damped utility occupancy value, the algorithm manages the last TID of scanned transactions (Line 05). After that, the data size and  $minSup$  are updated (Lines 06–07). If the user requests to find results, the *Restructure* process is performed to restructure the global DUOCA-List, and *Expansion* is conducted with the reconstructed global list (Lines 08–10). Finally, the result patterns are returned to the user (Line 11).

Algorithm 2 illustrates the sub-procedure for restructuring the global



**Table 4**

Characteristics of real data.

| Dataset  | Number of Transactions | Number of Items | T   <sub>AVG</sub> |
|----------|------------------------|-----------------|--------------------|
| Chess    | 3,196                  | 75              | 37                 |
| Mushroom | 8,124                  | 119             | 23                 |
| Pumsb    | 49,046                 | 7,116           | 73.6               |
| Retail   | 88,162                 | 16,469          | 10.3               |

**Table 5**

Characteristics of synthetic data.

| Dataset     | Number of Trans. | Number of Items | T   <sub>AVG</sub> |
|-------------|------------------|-----------------|--------------------|
| T10I4D200K  | 200,000          | 1,000           | 10                 |
| T10I4D400K  | 400,000          |                 |                    |
| T10I4D600K  | 600,000          |                 |                    |
| T10I4D800K  | 800,000          |                 |                    |
| T10I4D1000K | 1,000,000        |                 |                    |

list. It receives the global DUOCA-Lists and the last TID as parameters, and it returns a restructured global list. The global DUOCA-Lists are sorted by the ascending order of support for each item (Line 01). The proposed method iterates the nodes in the global list according to the reverse of the sorting order. Then, the method calculates the *ruo* of entries in each node (Lines 02–04). Afterward, the corresponding decaying value is calculated (Line 05). To calculate the *Sduo* and *Srduo* of each node, the *duo* and *rduo* of each entry in the node are added to *Sduo* and *Srduo*, respectively (Lines 06–07). Consequently, the restructured global list is returned (Line 08).

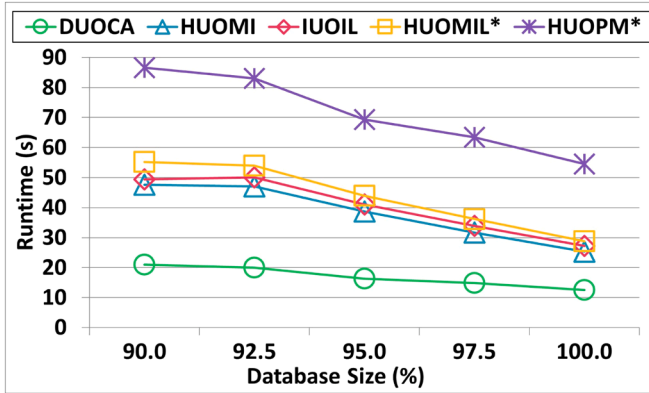
**Algorithm 3** outlines the process of extracting damped high utility occupancy patterns. In the *Expansion* process, the proposed method receives the DUOCA-List and a prefix pattern as parameters. This process is

recursively performed, and all patterns are extracted without loss or duplication. A result set, *HUOP*, is created and initialized (Line 01). The lists in the list of the current level are traversed (Lines 02–15). The pattern expansion continues if the support of the current list is no less than *minSup* (Lines 03–04). If the *Sduo* of  $C_i$  divided by the support of  $C_i$  is no less than threshold  $\beta$ , the pattern of  $C_i$  is combined with the prefix pattern, and it is included in *HUOP* (Lines 05–06). The *lubduo* and *ubduo* pruning strategies are performed (Line 07). When the two pruning strategies are satisfied, the conditional list of the next level, *NCL*, is generated (Line 08). Then, the node configuration of the extended patterns is conducted (Lines 09–12). In this process, the two lists  $C_i$  and  $C_j$  are combined for configuring entries, *Sduo*, and *Srduo* in the node of the extended pattern. If the pattern combination is ended, it is added to *NCL* (Lines 11–12). If there are two or more nodes in *NCL*, the *Expansion* calls itself recursively (Lines 13–14). When the *Expansion* is no longer called recursively, the result patterns are returned (Line 15).

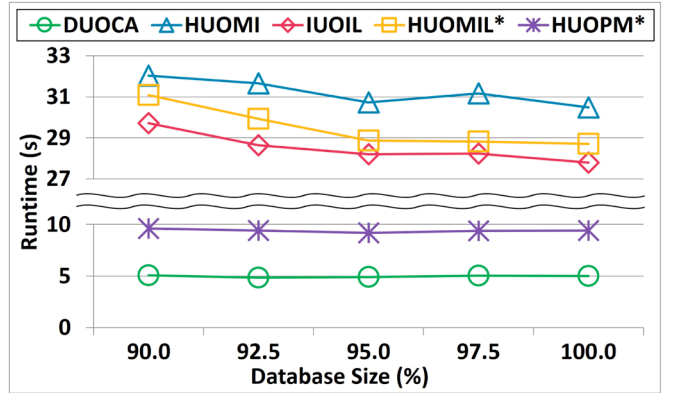
### 3.8. Computational complexity analysis of DUOCA

In the following section, an analysis of the computational complexity of DUOCA is illustrated. The analysis is composed of a comparison of the complexity between the proposed algorithm, DUOCA, and the state-of-the-art high utility pattern analysis approach with dynamic environments, HUOMI [45], which also uses a list structure. In the following analysis, assume that both approaches perform pattern extraction on the data with the following characteristics: the number of transactions,  $m$ , the number of unique items,  $n$ , and the average length of transactions in the data,  $k$ . Moreover, assume that reading and writing each item costs  $O(1)$ .

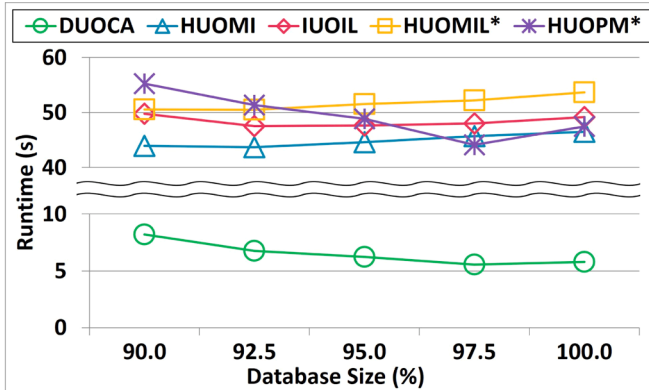
Both approaches perform the processes configured for constructing (updating), calculating sorting order, restructuring, and pattern expansion.



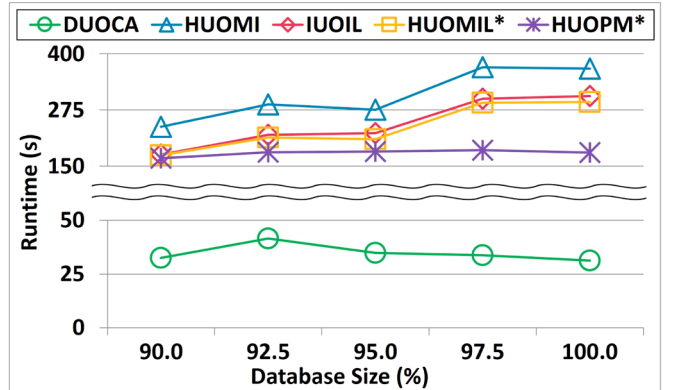
(a) Chess



(b) Mushroom



(c) Pumsb



(d) Retail

**Fig. 5.** Runtime performance on changing data size.

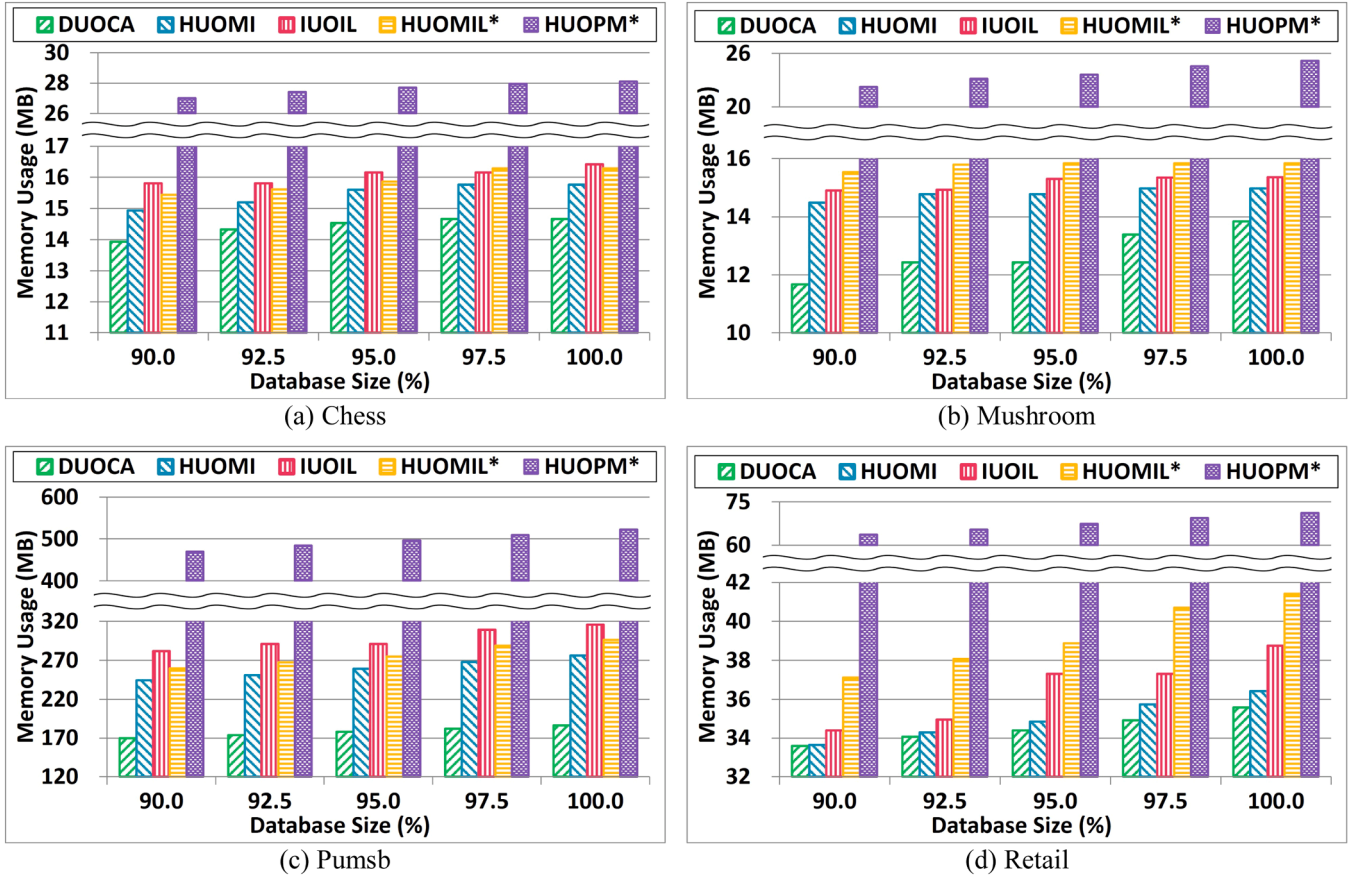


Fig. 6. Memory usage performance on changing data size.

sion to extract result patterns. In the constructing (updating) phase, since both algorithms handle the same data, they need to scan  $m \times k$  items, which requires the complexity of  $O(mk)$ . Then, to calculate the sorting order of the global list, both algorithms need to compare the support values of distinct items. This process costs  $O(n \log n)$  for both algorithms according to the optimized sorting process. Next, the restructuring process follows the calculation of the sorting order. This process requires a read, sort, and write operation for each transaction, which costs  $O(2k + k \log k)$ . Scaling it up to the entire data, the restructuring process costs  $O(2mk + mk \log k)$ . Since both algorithms handle equal work up to this point, their complexity is equal from the beginning to the restructuring procedure. Lastly, we address the calculation of the complexity of performing the pattern expansion procedure for each algorithm. The pattern expansion procedure accounts for a high proportion of the total process of pattern extraction, and the difference in complexity of pattern expansion is enough to lead to a difference in the overall complexity of the entire algorithm. The cost of the pattern expansion procedure is proportional to the size of the search space that each algorithm traverses to extract result patterns, which is proportional to the number of expandable patterns that have undergone the pruning strategy. To understand the scale of the search space in each algorithm, let the number of all extensible patterns be  $Cnt_{All}(S)$ , and the number of pruned items in DUOCA and HUOMI be  $Cnt_D(S, f)$  and  $Cnt_H(S)$ , respectively, where  $S$  represents the configuration settings regarding the data and the two thresholds. In both algorithms, the search space is determined by the upper bound values, configured as  $ubduo(X)$  in DUOCA and  $ubuo(X)$  in HUOMI for an arbitrary pattern  $X$ . The equation of  $ubuo(X)$  is denoted as follows:

$$ubuo(X) = \frac{\sum_{topminSup \wedge X \subseteq T_d} \{uo(X, T_d) + ruo(X, T_d)\}}{minSup} \quad (12)$$

When comparing Eq. (9) and Eq. (12),  $duo(X, T_d) \leq uo(X, T_d) = u(X, T_d) / tu(T_d)$  is always valid according to Definition 1, since  $f$  is a real number between 0 and 1. It means that  $ubduo(X)$  is a tighter lower bound than  $ubuo(X)$ , and the following relationship,  $Cnt_D(S, f) \geq Cnt_H(S)$ , is established, given the same data and threshold settings. Accordingly, the lower bound of DUOCA more effectively optimizes the search space in the pattern expansion procedure than that of HUOMI. Moreover, the value of  $Cnt_D(S, f)$  increases as the value of  $f$  gets smaller ( $0 < f < 1$ ), which leads to a larger difference in the search space considered by the two algorithms. As a result, it is demonstrated that the following relationship,  $Cnt_{All}(S) - Cnt_D(S, f) \leq Cnt_{All}(S) - Cnt_H(S)$ , is always valid. Therefore, DUOCA exhibits better computational complexity than HUOMI.

## 4. Performance evaluation

### 4.1. Data characteristics and experimental environment

This section shows the experimental environment and the data used for performance tests that were performed to evaluate the performance of DUOCA. We conducted comparisons with IUOIL [47], HUOMIL [46], HUOMI [45], and HUOPM [44]. IUOIL extracts high utility occupancy patterns in dynamic data, which cannot consider the damped window approach. HUOMIL is a state-of-the-art method in high utility occupancy pattern analysis, but since it was designed to operate on static data, we modified it to work in our tests with dynamic environments. HUOMI operates in dynamic environments but does not regard the damped window approach, unlike the proposed DUOCA. Lastly, HUOPM processes static data. Therefore, we gave it a dynamic method to process the gradually added data and also optimized it to utilize the damped factor for comparisons with DUOCA. The altered approaches are indicated

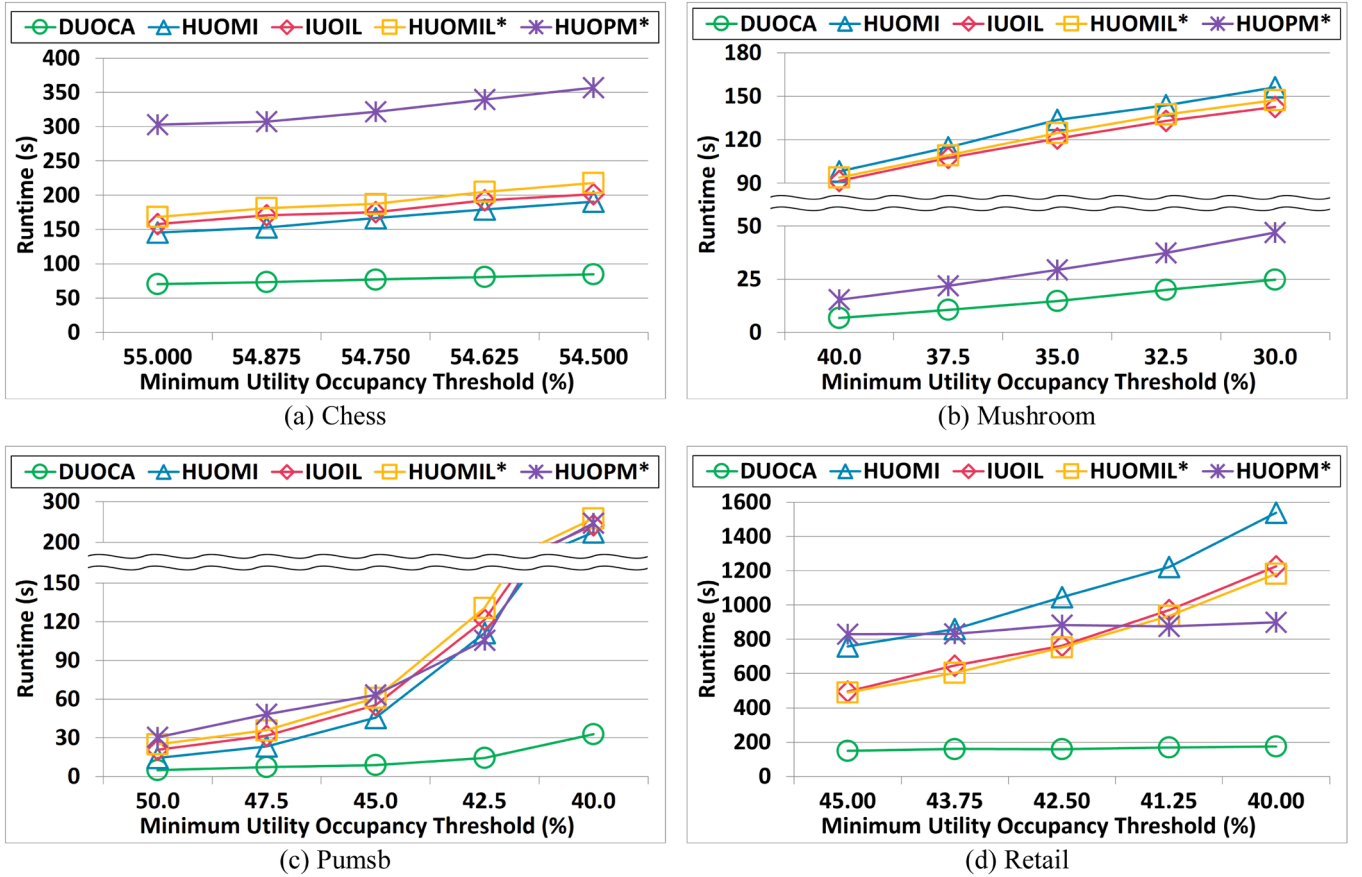


Fig. 7. Runtime performance on changing threshold.

with “\*”. All approaches were executed in the C/C++ language, and experimental tests were performed on PCs with the following configurations: Windows operating system, equipped with an Intel Core i7-6700 K processor (4.00 GHz) and 32 GB of RAM.

To demonstrate the superior performance of our algorithm, we performed tests using data with different characteristics. Tables 4 and 5 show the features of experimental data. The total number of items and transactions and the average transaction length vary in the real world, which is a significant factor in determining whether the data type is sparse or dense. Table 4, which shows the real data, accounts for the dense data, Chess, Mushroom, and Pumsb, and the sparse data, Retail. In several pattern analysis methods [50,52,53], such data have been used as benchmarking data and can be acquired from the FIMI repository (<http://fimi.ua.ac.be/data/>). Furthermore, we utilized a set of synthetic data with dense features that were created with the IBM Data Generator. As shown in Table 5, the count of items and the average length of transactions are constant, but the total transaction count increases continuously.

#### 4.2. Runtime and memory usage test on dynamic steps

We present the first experimental measurement of memory utilization and runtime of the methods. To set up an environment where data streams consistently occur, real data are divided into five parts and are entered into all algorithms in order. The first part, which holds 90% of the entire data, is configured with the original data stream, and the remainders, divided into 2.5% dynamic data streams, signify the additionally inserted data. The experiment introduced in this section was conducted with both the minimum utility occupancy threshold ( $\beta$ ) and minimum support threshold ( $\alpha$ ) fixed to specific values, respectively. Figs. 5 and 6 illustrate the measured runtime and peak memory

consumption results. Fig. 5(a)–(d) illustrate the runtime results for Chess ( $\alpha=20\%$ ,  $\beta=54.5\%$ ), Mushroom ( $\alpha=12.5\%$ ,  $\beta=30\%$ ), Pumsb ( $\alpha=45\%$ ,  $\beta=40\%$ ), and Retail ( $\alpha=0.002\%$ ,  $\beta=40\%$ ), respectively. Additionally,  $f$  was set to 0.999975, 0.999747, 0.999992, and 0.999952 for the data, respectively. In Fig. 5(a), Chess, the runtime of the algorithms decreases as the data size increases, and DUOCA completes the fastest, especially at x-axis points 90% and 92.5% where the runtime is relatively high compared to the other points. In Fig. 5(d), the runtime results of IUOIL, HUOMIL\* and HUOMI increase as the data accumulates. Conversely, DUOCA and HUOPM\* show stable runtime results, which demonstrate the effects of the damped window control. Fig. 5(b) and (c) exhibit the runtime results of the tests affected by data insertions. The results also show that the decaying factor is more effective in Retail and Mushroom, as HUOPM\* utilizing the damped window control is relatively faster on those data than other state-of-the-art methods. Nevertheless, DUOCA finished fastest in all tested data due to search space reduction through damped window control-based pruning and efficient list operations. Fig. 6 shows the memory consumption of each algorithm for four real data measured in the same settings as Fig. 5. In these results, the memory usage tends to increase, unlike Fig. 5. The reason why the trend is shown in most results is that as the data scale rises, the count of patterns managed in memory also increases. Despite this, DUOCA is always better than or comparable to the other comparison methods. For instance, DUOCA uses close to but less than 5% less memory usage than HUOMI at the data size of 100% in Chess. These merits, along with the runtime results, demonstrate that the proposed method performs efficiently in environments where data consistently occur.

#### 4.3. Runtime and memory usage test on varying thresholds

The results of memory and runtime measurements, derived from



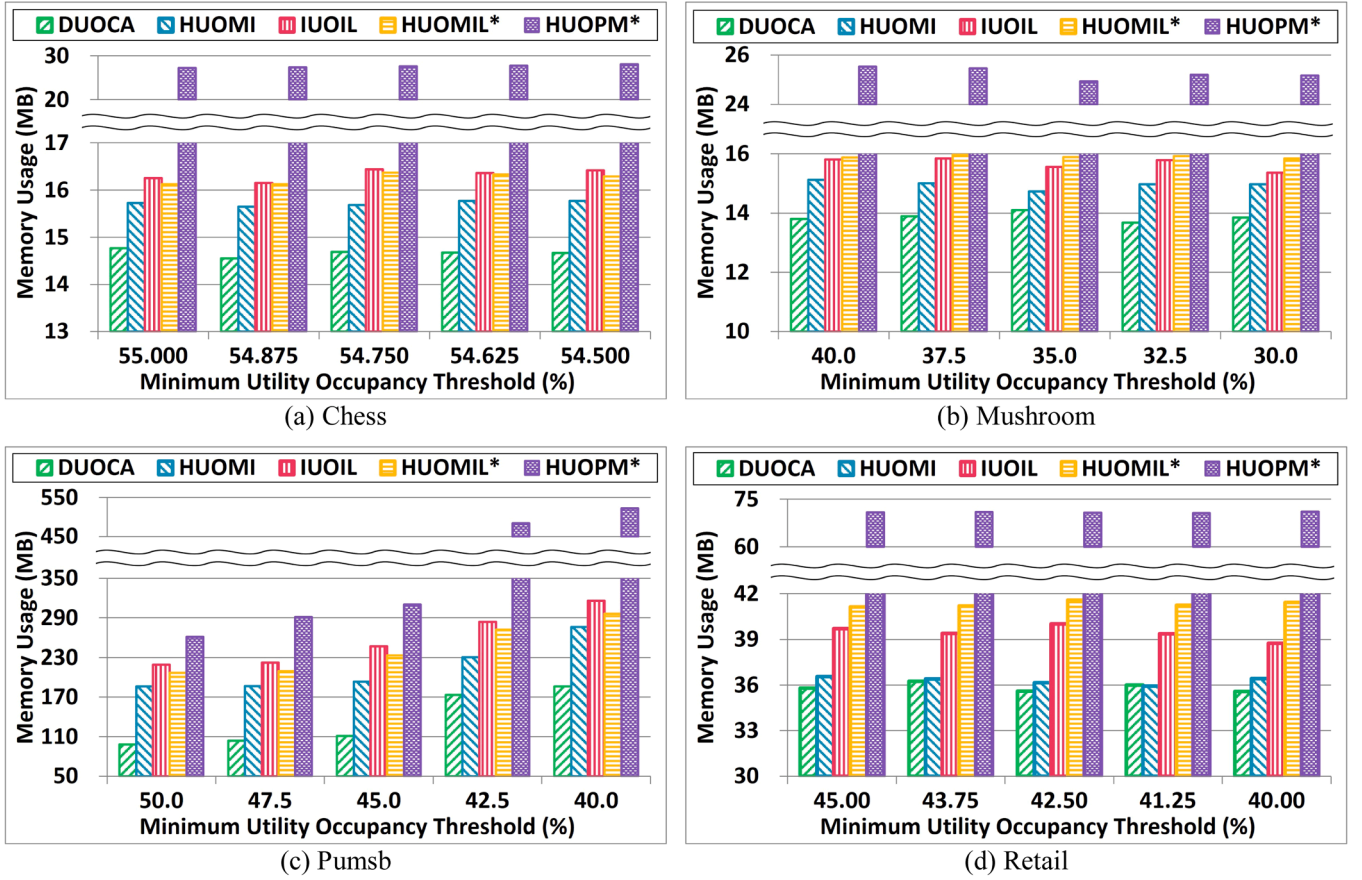


Fig. 8. Memory usage performance on changing threshold.

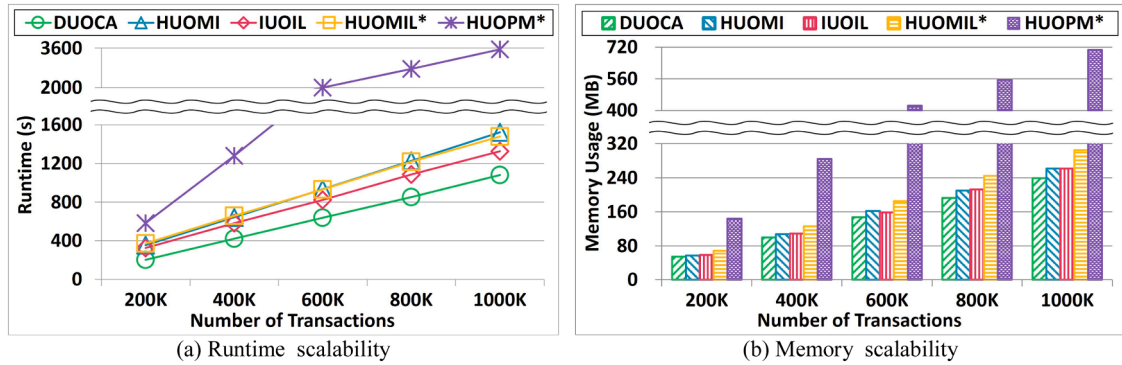


Fig. 9. Runtime and memory scalability performance.

changing minimum utility occupancy thresholds, are described in this section. All five algorithms, DUOCA, IUOIL, HUOMIL\*, HUOMI, and HUOPM\*, utilize the two thresholds of minimum support threshold and minimum utility occupancy threshold, denoted as  $\alpha$  and  $\beta$ , respectively. For the tests in this section,  $\alpha$  was fixed, and  $\beta$  gradually increased. Moreover, the five parts of real data division for measurement in dynamic environments, as well as the  $f$  value, are configured in the same way as described in Section 4.2. Fig. 7(a)–(d) show the experimental results for the runtime evaluations. In general, runtime increases as  $\beta$  decreases because the number of result patterns increases. This trend is evident in all data, but DUOCA exhibits the most efficient runtime performance even with the condition. In fact, DUOCA is at least 20%, 47%, 85%, and 80% faster than all other methods at the lowest  $\beta$  in Chess, Pumsb, Mushroom, and Retail, respectively. Following the trends of the results in Section 4.2, DUOCA and HUOPM\* are more efficient

than IUOIL, HUOMIL\* and HUOMI in Fig. 7(b) and (d), Mushroom and Retail, as the threshold decreases, due to the damped window control-based pruning effect. Nonetheless, in all four real data, DUOCA is always superior to the other four methods in terms of runtime, and the runtime difference between the state-of-the-art algorithms tends to increase as  $\beta$  decreases. This demonstrates the strengths of the damped window control of the proposed algorithm regarding runtime. Fig. 8(a)–(d) illustrate the memory usage results in the condition where  $\alpha$  and  $\beta$  are equally set to the experiments in Fig. 6. The results show a similar tendency to those from Section 4.2. In particular, the proposed DUOCA is better than or comparable to the other state-of-the-art methods regarding memory usage results. It is also evident that peak memory usage is also affected by the change in  $\beta$  compared to runtime.



**Table 6**

Runtime and Memory usage result of ablation study on pruning strategies.

| Dataset  | Method  | Minimum utility occupancy threshold $\beta$ (%) |               |               |               |               |                   |               |               |               |               |
|----------|---------|-------------------------------------------------|---------------|---------------|---------------|---------------|-------------------|---------------|---------------|---------------|---------------|
|          |         | Runtime (s)                                     |               |               |               |               | Memory usage (MB) |               |               |               |               |
| Chess    |         | 0.55000                                         | 0.54875       | 0.54750       | 0.54625       | 0.54500       | 0.55000           | 0.54875       | 0.54750       | 0.54625       | 0.54500       |
|          | DUOCA   | <b>56.42</b>                                    | <b>58.88</b>  | <b>62.07</b>  | <b>65.54</b>  | <b>69.19</b>  | 15.31             | 15.59         | <b>15.43</b>  | <b>15.52</b>  | 15.77         |
|          | DUOCA-1 | 9243.50                                         | 9462.35       | 9369.33       | 9463.38       | 9555.23       | 17.07             | 17.18         | 17.28         | 17.07         | 17.57         |
|          | DUOCA-2 | 123.72                                          | 129.96        | 137.97        | 144.08        | 152.22        | <b>15.27</b>      | <b>15.09</b>  | 15.56         | 15.52         | <b>15.47</b>  |
| Mushroom |         | 40.0                                            | 37.5          | 35.0          | 32.5          | 30.0          | 40.0              | 37.5          | 35.0          | 32.5          | 30.0          |
|          | DUOCA   | <b>6.09</b>                                     | 9.29          | 12.95         | 17.20         | 21.66         | <b>14.29</b>      | <b>14.39</b>  | <b>14.74</b>  | 14.64         | 14.70         |
|          | DUOCA-1 | 7.72                                            | <b>8.79</b>   | <b>10.25</b>  | <b>12.53</b>  | <b>15.66</b>  | 14.76             | 14.89         | 15.29         | 15.22         | 15.40         |
|          | DUOCA-2 | 7.16                                            | 10.38         | 14.19         | 18.80         | 23.54         | 14.57             | 15.05         | 14.78         | <b>14.49</b>  | <b>14.36</b>  |
| Pumsb    |         | 50.0                                            | 47.5          | 45.0          | 42.5          | 40.0          | 50.0              | 47.5          | 45.0          | 42.5          | 40.0          |
|          | DUOCA   | <b>4.28</b>                                     | <b>6.16</b>   | <b>7.48</b>   | <b>12.23</b>  | <b>27.57</b>  | <b>99.83</b>      | <b>107.78</b> | <b>113.32</b> | 176.34        | <b>190.10</b> |
|          | DUOCA-1 | 14.91                                           | 27.75         | 45.21         | 70.01         | 115.31        | 204.25            | 217.67        | 219.07        | 232.51        | 249.53        |
|          | DUOCA-2 | 8.52                                            | 15.88         | 19.23         | 37.25         | 96.05         | 100.40            | 108.40        | 114.22        | <b>175.50</b> | 190.84        |
| Retail   |         | 45.00                                           | 43.75         | 42.50         | 41.25         | 40.00         | 45.00             | 43.75         | 42.50         | 41.25         | 40.00         |
|          | DUOCA   | <b>145.19</b>                                   | <b>152.26</b> | <b>155.00</b> | <b>160.89</b> | <b>168.48</b> | 36.08             | 36.55         | 36.70         | 36.56         | 36.84         |
|          | DUOCA-1 | 177.29                                          | 179.36        | 184.10        | 188.73        | 194.36        | <b>35.49</b>      | <b>35.25</b>  | <b>35.47</b>  | <b>36.15</b>  | <b>35.27</b>  |
|          | DUOCA-2 | 156.46                                          | 162.55        | 167.14        | 177.28        | 185.20        | 36.54             | 36.42         | 36.79         | 36.41         | 36.40         |

**Table 7**

Ablation study result on the effect of decaying factor parameters.

| Dataset                                           | Metric      | Method                        |                |                |               |                |
|---------------------------------------------------|-------------|-------------------------------|----------------|----------------|---------------|----------------|
| Chess<br>( $\alpha = 20\%$ , $\beta = 54.5\%$ )   |             | DUOCA (decaying factor, $f$ ) |                |                |               |                |
|                                                   |             | 0.99991                       | 0.99992        | 0.99993        | 0.99994       | 0.99995        |
|                                                   | Runtime (s) | <b>11.52</b>                  | <b>14.379</b>  | <b>18.492</b>  | <b>24.827</b> | <b>38.165</b>  |
|                                                   | Memory (MB) | <b>12.902</b>                 | <b>13.348</b>  | <b>13.863</b>  | <b>13.988</b> | <b>14.258</b>  |
| Mushroom<br>( $\alpha = 10\%$ , $\beta = 30\%$ )  |             | DUOCA (decaying factor, $f$ ) |                |                |               |                |
|                                                   |             | 0.99981                       | 0.99982        | 0.99983        | 0.99984       | 0.99985        |
|                                                   | Runtime (s) | <b>34.18</b>                  | <b>37.18</b>   | <b>41.473</b>  | <b>46.134</b> | <b>53.04</b>   |
|                                                   | Memory (MB) | <b>15.219</b>                 | <b>15.23</b>   | <b>15.09</b>   | <b>15.441</b> | <b>15.535</b>  |
| Pumsb<br>( $\alpha = 45\%$ , $\beta = 40\%$ )     |             | DUOCA (decaying factor, $f$ ) |                |                |               |                |
|                                                   |             | 0.999991                      | 0.999992       | 0.999993       | 0.999994      | 0.999995       |
|                                                   | Runtime (s) | <b>25.004</b>                 | <b>33.729</b>  | <b>44.493</b>  | <b>53.879</b> | <b>80.525</b>  |
|                                                   | Memory (MB) | <b>190.238</b>                | <b>190.602</b> | <b>190.633</b> | <b>194.68</b> | <b>208.648</b> |
| Retail<br>( $\alpha = 0.002\%$ , $\beta = 40\%$ ) |             | DUOCA (decaying factor, $f$ ) |                |                |               |                |
|                                                   |             | 0.99981                       | 0.99982        | 0.99983        | 0.99984       | 0.99985        |
|                                                   | Runtime (s) | <b>98.316</b>                 | <b>99.957</b>  | <b>102.214</b> | <b>107.54</b> | <b>108.627</b> |
|                                                   | Memory (MB) | <b>36.273</b>                 | <b>36.645</b>  | <b>36.301</b>  | <b>36.691</b> | <b>36.656</b>  |

#### 4.4. Scalability test

In this part, we analyze the usage of memory and runtime performance of the approaches by employing the synthetic data in Table 5. T1014DxK, the data group used in this test, comprises five dense data where  $x$  is a value increasing from 200 K to 1000 K. Note that a set of data has the same quantity of unique items and average length of transactions, but not the total of transactions. Fig. 9(a) and (b) illustrate the runtime and memory measurements, respectively, with  $\alpha=0.01\%$  and  $\beta=1\%$ . Basically, as the size of the data grows, the consumption of resources such as memory and runtime also increases. As shown in Fig. 9 (a), the slope for the runtime increase of DUOCA was gentler than those of HUOMIL\*, HUOMI, and HUOPM\*. When considering the actual runtime results, DUOCA, IUOIL, HUOMIL\*, HUOMI, and HUOPM\* took approximately 202, 325, 373, 354, and 582 seconds at  $x=200$  K and 1,082, 1,326, 1,478, 1,521, and 3,550 seconds at  $x=1000$  K, respectively. The memory usage of DUOCA, shown in Fig. 9(b), exhibits better performance than the other methods regardless of the  $x$  value. For instance, at  $x=1000$  K, the five algorithms, in order, consumed around 239, 262, 304, 262, and 705 MB, respectively. Therefore, these experimental results demonstrate that DUOCA has better scalability compared to the other state-of-the-art methods in terms of runtime and memory usage.

#### 4.5. Ablation study for analyzing the contribution of individual components

##### 4.5.1. Ablation study on the proposed pruning strategies

This section presents an ablation study to evaluate the performance impact of the proposed pruning strategies in DUOCA. Specifically, the performance of the DUOCA is compared with two modified methods: DUOCA-1, which is DUOCA with the *lubduo* strategy, and DUOCA-2, which is DUOCA with the *ubduo* strategy. Table 6 demonstrates the runtime and memory usage of all three methods across four datasets under the experimental setup described in Section 4.3. As shown in the table, for all methods, the runtime tends to increase as the minimum utility occupancy threshold  $\beta$  decreases. This is due to the larger number of patterns being extracted, which results in a larger search space and greater computational cost. Overall, DUOCA, which integrates both pruning strategies, demonstrates better runtime performance than the other methods. For instance, on the Pumsb dataset with  $\beta = 40\%$ , DUOCA-1 and DUOCA-2 required 115.3 and 96.0 seconds, respectively, whereas DUOCA required 27.56 seconds. Similarly, on the Retail dataset with  $\beta = 40\%$ , DUOCA achieves a runtime of 168.5 seconds, compared to 194.4 and 185.2 seconds for DUOCA-1 and DUOCA-2, respectively. On the Mushroom dataset, DUOCA-1 exhibits slightly better runtime than DUOCA and DUOCA-2 at lower  $\beta$  values, indicating that *lubduo* provides a more effective pruning effect than *ubduo* in this case. With respect to memory usage, DUOCA, DUOCA-1, and DUOCA-2 demonstrate comparable or competitive performance. In conclusion, the ablation study indicates the effectiveness of the proposed pruning

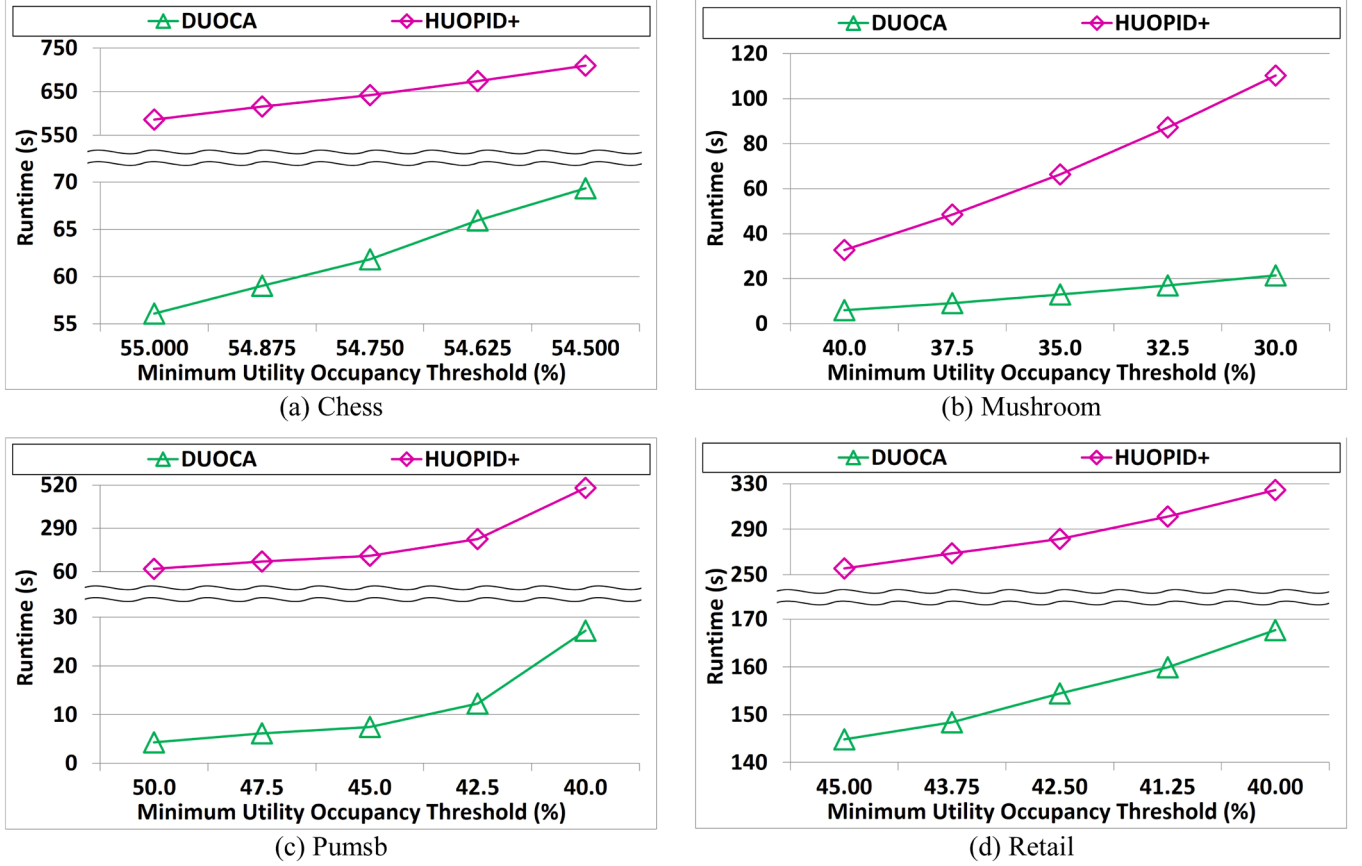


Fig. 10. Runtime comparison of list-based and tree-based methods.

strategies in DUOCA, demonstrating overall improvements in runtime performance while maintaining similar memory usage.

#### 4.5.2. Ablation study on damped window with varying the decaying factor

This section presents an analysis of the algorithmic efficiency of DUOCA by incorporating the time-sensitive nature of the data. To assess the efficiency of DUOCA, a series of experiments were conducted with varying decaying factors. This experimental design was intended to demonstrate how the damped window control embedded in DUOCA contributes to improving computational efficiency. Table 7 summarizes the experimental results obtained by applying different decaying factors to DUOCA across four datasets. For comparison, the performance of HUOMI, which serves as a baseline method that does not take temporal sensitivity into account, was also shown. The results indicate that DUOCA consistently outperforms HUOMI in terms of runtime across all datasets. Within DUOCA itself, a smaller decaying factor generally leads to shorter runtimes. This trend is attributed to the fact that, under the same utility threshold, smaller decaying factors result in reduced support and occupancy values of patterns, thereby increasing the likelihood of pruning during the mining process. Furthermore, the performance gains diminish progressively as the decaying factor continues to decrease. For instance, in the Chess dataset, DUOCA achieved approximately three times faster runtime and a 10% improvement in memory efficiency when the decaying factor was set to 0.99991 compared to 0.99995. However, the reduction in runtime was 53% when moving from 0.99995 to 0.99994, whereas it decreased by 24% when transitioning from 0.99992 to 0.99991. This observation suggests that the marginal impact of decaying factor adjustments becomes less pronounced as the factor becomes smaller. In terms of memory usage, the Chess and Pumsb datasets exhibited a decreasing trend as the decaying factor was reduced. In contrast, the Mushroom and Retail datasets

showed minimal variation in memory consumption. These results imply that DUOCA is capable of extracting patterns effectively without incurring additional memory overhead. Overall, the experimental findings demonstrate that DUOCA, by integrating the time-sensitivity strategy, is capable of performing pattern extraction in a computationally efficient manner while maintaining effectiveness in handling time-sensitive data.

#### 4.5.3. Ablation study on data structures

The ablation test was conducted on four datasets, which are Chess, Mushroom, Pumsb, and Retail, with five minimum utility occupancy thresholds for each dataset. The results were compared and analyzed in terms of runtime and memory usage. Figs. 10 and 11 illustrate the experimental results by comparing the proposed method and the baseline method utilizing a tree-based structure. Across all datasets and threshold settings, DUOCA consistently recorded lower execution times than HUOPID+. In particular, as the number of items in the dataset increased and the threshold decreased, the runtime of HUOPID+ increased significantly, whereas DUOCA maintained a relatively stable runtime. In terms of memory usage, DUOCA consumed less memory than HUOPID+. Across all datasets, DUOCA achieved memory savings of at least two times and up to six times. This difference was particularly pronounced on relatively large-scale datasets, including Pumsb and Retail, compared to Chess and Mushroom. DUOCA outperformed HUOPID+ in both runtime and memory usage. In particular, its consistent superiority under various conditions highlights the structural efficiency and stability of DUOCA.

#### 4.6. Significance test

In this section, we statistically demonstrate that the proposed

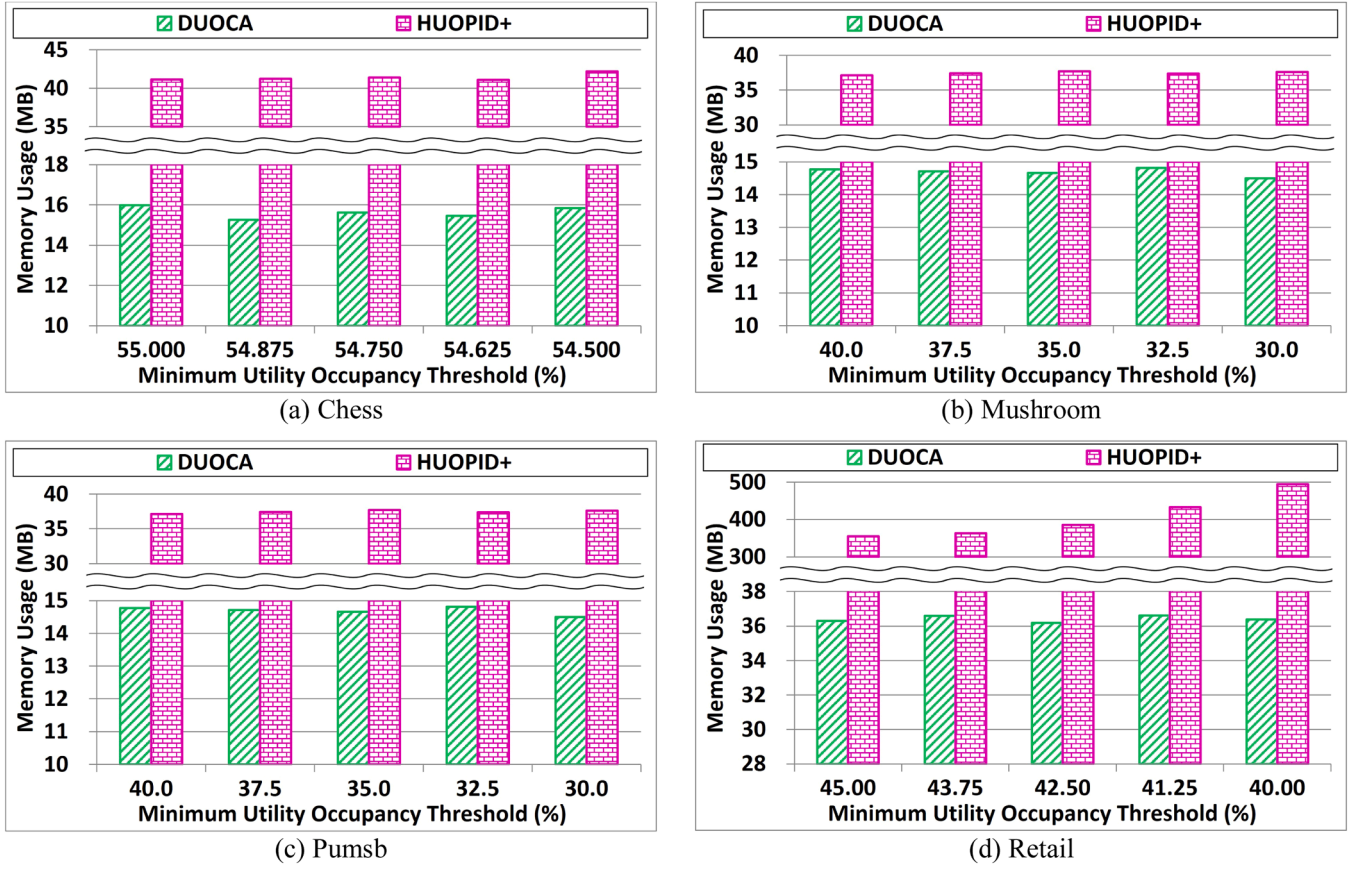


Fig. 11. Memory usage comparison of list-based and tree-based methods.

Table 8

Significance Test on 30 sampled real data.

| Chess (30 Sampled)    | Mean       | Variance    | SE         | Z-score    | P-value |
|-----------------------|------------|-------------|------------|------------|---------|
| DUOCA                 | 47587.5    | 1339653.983 | —          | —          | —       |
| IUOIL                 | 96560.467  | 7374557.706 | 538.95614  | 90.86633   | 0.00000 |
| HUOMIL*               | 96560.467  | 7374557.706 | 538.95614  | 90.86633   | 0.00000 |
| HUOMI                 | 96560.467  | 7374557.706 | 538.95614  | 90.86633   | 0.00000 |
| Mushroom (30 Sampled) | Mean       | Variance    | SE         | Z-score    | P-value |
| DUOCA                 | 12423.233  | 25.082      | —          | —          | —       |
| IUOIL                 | 100320.567 | 25577.289   | 29.21322   | 3008.82023 | 0.00000 |
| HUOMIL*               | 100320.567 | 25577.289   | 29.21322   | 3008.82023 | 0.00000 |
| HUOMI                 | 100320.567 | 25577.289   | 29.21322   | 3008.82023 | 0.00000 |
| Pumsb (30 Sampled)    | Mean       | Variance    | SE         | Z-score    | P-value |
| DUOCA                 | 504.933    | 0.47816     | —          | —          | —       |
| IUOIL                 | 1423.8     | 10.441      | 0.60331    | 1523.03873 | 0.00000 |
| HUOMIL*               | 1423.8     | 10.441      | 0.60331    | 1523.03873 | 0.00000 |
| HUOMI                 | 1423.8     | 10.441      | 0.60331    | 1523.03873 | 0.00000 |
| Retail (30 Sampled)   | Mean       | Variance    | SE         | Z-score    | P-value |
| DUOCA                 | 911502.2   | 2161881.5   | —          | —          | —       |
| IUOIL                 | 2153875.4  | 28012123.3  | 1002.89589 | 1238.78585 | 0.00000 |
| HUOMIL*               | 2153875.4  | 28012123.3  | 1002.89589 | 1238.78585 | 0.00000 |
| HUOMI                 | 2153875.4  | 28012123.3  | 1002.89589 | 1238.78585 | 0.00000 |

approach employing the damped window control to assign the relative importance of each transaction extracts refined result patterns. We conducted a significance test [51] with IUOIL, HUOMIL\* and HUOMI for comparisons, which do not consider the temporal aspect of each pattern. The test proceeds as follows: First, a null hypothesis claiming that the proposed method has no impact on reducing the extracted pattern set is defined. Second, for trustworthy results, 30 datasets are sampled for each real dataset utilized in our previous experiments through the swap randomization technique utilized in recent studies [26,51]. Third, for each created dataset group, the means and variances of the number of

visited patterns included in result patterns are recorded for each algorithm. Fourth, the null hypothesis is rejected by performing the z-test and calculating the p-value, thereby demonstrating the statistical significance of DUOCA. The z-score for the z-test is calculated as follows, where  $\sigma_1^2(\sigma_2^2)$ ,  $n_1(n_2)$ , and  $\bar{Y}_1(\bar{Y}_2)$  represent the variances, numbers of samples, and means of the compared algorithms, respectively:

$$SE = \sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}} \quad (13)$$

**Table 9**

Runtime result of concept-drifting test with real datasets.

| Chess ( $\alpha = 20\%$ )     |         |         |         |         |         |             |         |         |         |         |         |
|-------------------------------|---------|---------|---------|---------|---------|-------------|---------|---------|---------|---------|---------|
| $\beta$                       | 60%     | 59.5%   | 59%     | 58.5%   | 58%     | $\beta$     | 60%     | 59.5%   | 59%     | 58.5%   | 58%     |
| DUOCA-F75                     | 37.403  | 46.832  | 56.099  | 71.537  | 96.19   | IUOIL-F75   | 358.003 | 481.74  | 659.485 | 888.203 | 1209.01 |
| DUOCA-F150                    | 20.321  | 25.964  | 31.939  | 40.835  | 52.719  | IUOIL-F150  | 203.126 | 282.466 | 383.354 | 528.878 | 683.135 |
| DUOCA-F300                    | 12.853  | 16.064  | 20.572  | 26.614  | 34.172  | IUOIL-F300  | 132.828 | 184.315 | 243.382 | 333.672 | 437.949 |
| DUOCA-F500                    | 9.985   | 12.43   | 16.673  | 21.248  | 27.378  | IUOIL-F500  | 104.148 | 143.751 | 194.76  | 260.676 | 343.322 |
| DUOCA-F750                    | 8.83    | 11.02   | 14.262  | 18.788  | 25.989  | IUOIL-F750  | 95.972  | 131.844 | 177.27  | 242.28  | 327.463 |
| Mushroom ( $\alpha = 10\%$ )  |         |         |         |         |         |             |         |         |         |         |         |
| $\beta$                       | 60%     | 57.5%   | 55%     | 52.5%   | 50%     | $\beta$     | 60%     | 57.5%   | 55%     | 52.5%   | 50%     |
| DUOCA-F125                    | 9.471   | 14.861  | 28.834  | 58.267  | 116.756 | IUOIL-F125  | 183.131 | 271.948 | 386.779 | 528.259 | 683.048 |
| DUOCA-F250                    | 4.978   | 8.243   | 15.998  | 32.159  | 61.975  | IUOIL-F250  | 97.121  | 145.976 | 204.399 | 279.64  | 358.418 |
| DUOCA-F500                    | 2.827   | 4.75    | 9.368   | 18.578  | 35.578  | IUOIL-F500  | 53.301  | 79.636  | 112.87  | 152.832 | 197.019 |
| DUOCA-F1000                   | 1.714   | 3.09    | 6.021   | 11.857  | 22.244  | IUOIL-F1000 | 32.52   | 47.616  | 68.087  | 93.905  | 119.016 |
| DUOCA-F2000                   | 1.287   | 2.391   | 4.952   | 9.839   | 18.275  | IUOIL-F2000 | 21.975  | 32.775  | 46.869  | 63.934  | 82.876  |
| Pumsb ( $\alpha = 45\%$ )     |         |         |         |         |         |             |         |         |         |         |         |
| $\beta$                       | 50%     | 47.5%   | 45%     | 42.5%   | 40%     | $\beta$     | 50%     | 47.5%   | 45%     | 42.5%   | 40%     |
| DUOCA-F1K                     | 36.331  | 62.056  | 119.706 | 210.506 | 377.495 | IUOIL-F1K   | 104.373 | 154.358 | 240.444 | 407.175 | 723.134 |
| DUOCA-F2K                     | 19.223  | 33.525  | 65.985  | 115.433 | 197.392 | IUOIL-F2K   | 56.103  | 83.332  | 131.860 | 219.181 | 379.812 |
| DUOCA-F4K                     | 11.279  | 18.586  | 36.273  | 62.289  | 109.810 | IUOIL-F4K   | 30.834  | 47.331  | 72.768  | 125.149 | 214.666 |
| DUOCA-F8K                     | 6.485   | 11.492  | 22.227  | 38.843  | 68.129  | IUOIL-F8K   | 19.056  | 28.198  | 45.649  | 77.741  | 132.528 |
| DUOCA-F16K                    | 4.721   | 9.571   | 19.057  | 33.772  | 56.653  | IUOIL-F16K  | 15.784  | 23.103  | 36.360  | 62.373  | 102.042 |
| Retail ( $\alpha = 0.002\%$ ) |         |         |         |         |         |             |         |         |         |         |         |
| $\beta$                       | 40%     | 38.75%  | 37.5%   | 36.25%  | 35%     | $\beta$     | 40%     | 38.75%  | 37.5%   | 36.25%  | 35%     |
| DUOCA-F1K                     | 1306.58 | 1471.76 | 1780.29 | 2149.57 | 2699.64 | IUOIL-F1K   | 2065.87 | 2645.20 | 3399.03 | 4296.56 | 5424.84 |
| DUOCA-F2K                     | 665.212 | 779.287 | 941.43  | 1145.92 | 1419.10 | IUOIL-F2K   | 1090.96 | 1404.47 | 1794.94 | 2301.35 | 2902.47 |
| DUOCA-F4K                     | 370.173 | 436.786 | 533.743 | 639.49  | 794.853 | IUOIL-F4K   | 608.829 | 785.268 | 1003.41 | 1283.92 | 1619.80 |
| DUOCA-F8K                     | 230.411 | 271.804 | 328.902 | 401.213 | 498.122 | IUOIL-F8K   | 382.464 | 487.62  | 628.97  | 797.219 | 1020.40 |
| DUOCA-F16K                    | 165.593 | 183.106 | 220.031 | 270.215 | 342.687 | IUOIL-F16K  | 263.324 | 327.933 | 421.043 | 535.623 | 691.004 |

**Table 10**

Memory usage results of concept-drifting test with real datasets.

| Chess ( $\alpha = 20\%$ )     |         |         |         |         |         |             |         |         |         |         |         |
|-------------------------------|---------|---------|---------|---------|---------|-------------|---------|---------|---------|---------|---------|
| $\beta$                       | 60%     | 59.5%   | 59%     | 58.5%   | 58%     | $\beta$     | 60%     | 59.5%   | 59%     | 58.5%   | 58%     |
| DUOCA-F75                     | 10.906  | 11.121  | 11.043  | 11.859  | 12.004  | IUOIL-F75   | 13.391  | 13.344  | 12.980  | 13.316  | 13.340  |
| DUOCA-F150                    | 10.355  | 10.914  | 10.941  | 11.531  | 11.273  | IUOIL-F150  | 13.230  | 13.199  | 13.195  | 13.250  | 13.137  |
| DUOCA-F300                    | 10.344  | 10.742  | 10.609  | 10.910  | 11.492  | IUOIL-F300  | 13.016  | 12.953  | 12.984  | 12.965  | 13.059  |
| DUOCA-F500                    | 10.109  | 10.723  | 10.688  | 11.406  | 11.215  | IUOIL-F500  | 12.934  | 12.953  | 12.984  | 12.992  | 12.938  |
| DUOCA-F750                    | 10.656  | 10.371  | 10.652  | 10.996  | 11.344  | IUOIL-F750  | 12.961  | 13.020  | 12.957  | 12.926  | 12.898  |
| Mushroom ( $\alpha = 10\%$ )  |         |         |         |         |         |             |         |         |         |         |         |
| $\beta$                       | 60%     | 57.5%   | 55%     | 52.5%   | 50%     | $\beta$     | 60%     | 57.5%   | 55%     | 52.5%   | 50%     |
| DUOCA-F125                    | 13.707  | 14.867  | 15.000  | 15.082  | 15.684  | IUOIL-F125  | 17.156  | 16.984  | 17.063  | 17.094  | 17.262  |
| DUOCA-F250                    | 13.754  | 15.059  | 15.004  | 15.539  | 15.234  | IUOIL-F250  | 16.625  | 16.719  | 17.051  | 16.672  | 16.855  |
| DUOCA-F500                    | 13.707  | 14.801  | 14.684  | 14.961  | 15.313  | IUOIL-F500  | 16.922  | 17.027  | 16.684  | 16.594  | 16.660  |
| DUOCA-F1000                   | 13.531  | 14.414  | 15.277  | 14.852  | 14.938  | IUOIL-F1000 | 16.918  | 16.648  | 16.570  | 16.398  | 16.453  |
| DUOCA-F2000                   | 13.004  | 14.410  | 14.621  | 14.340  | 14.410  | IUOIL-F2000 | 15.918  | 16.012  | 16.168  | 16.250  | 16.195  |
| Pumsb ( $\alpha = 45\%$ )     |         |         |         |         |         |             |         |         |         |         |         |
| $\beta$                       | 50%     | 47.5%   | 45%     | 42.5%   | 40%     | $\beta$     | 50%     | 47.5%   | 45%     | 42.5%   | 40%     |
| DUOCA-F1K                     | 122.969 | 126.227 | 131.418 | 135.449 | 154.996 | IUOIL-F1K   | 177.176 | 181.836 | 199.754 | 236.246 | 260.582 |
| DUOCA-F2K                     | 121.680 | 125.363 | 130.219 | 136.320 | 155.000 | IUOIL-F2K   | 175.609 | 180.699 | 195.340 | 230.871 | 251.371 |
| DUOCA-F4K                     | 120.813 | 124.707 | 129.328 | 135.254 | 152.297 | IUOIL-F4K   | 174.039 | 179.156 | 187.918 | 220.496 | 250.297 |
| DUOCA-F8K                     | 120.941 | 124.492 | 127.719 | 135.371 | 151.332 | IUOIL-F8K   | 172.262 | 178.730 | 185.547 | 216.195 | 252.406 |
| DUOCA-F16K                    | 121.766 | 115.113 | 118.727 | 128.293 | 152.828 | IUOIL-F16K  | 171.004 | 168.523 | 174.617 | 206.250 | 249.383 |
| Retail ( $\alpha = 0.002\%$ ) |         |         |         |         |         |             |         |         |         |         |         |
| $\beta$                       | 40%     | 38.75%  | 37.5%   | 36.25%  | 35%     | $\beta$     | 40%     | 38.75%  | 37.5%   | 36.25%  | 35%     |
| DUOCA-F1K                     | 34.523  | 33.941  | 34.004  | 33.578  | 34.109  | IUOIL-F1K   | 48.328  | 48.176  | 48.285  | 48.203  | 47.938  |
| DUOCA-F2K                     | 33.949  | 34.059  | 33.898  | 34.000  | 34.016  | IUOIL-F2K   | 45.230  | 44.223  | 44.898  | 44.547  | 44.578  |
| DUOCA-F4K                     | 33.559  | 33.727  | 33.391  | 33.547  | 33.359  | IUOIL-F4K   | 41.930  | 42.266  | 41.738  | 41.504  | 42.219  |
| DUOCA-F8K                     | 32.945  | 33.430  | 33.105  | 33.230  | 33.031  | IUOIL-F8K   | 40.020  | 39.371  | 39.895  | 40.309  | 39.773  |
| DUOCA-F16K                    | 33.535  | 33.078  | 33.910  | 33.641  | 33.051  | IUOIL-F16K  | 38.988  | 38.852  | 39.352  | 38.813  | 38.902  |

$$z - score = \frac{|\bar{Y}_1 - \bar{Y}_2|}{SE} \quad (14)$$

Table 8 displays the results of the significance test. The experimental parameters,  $\alpha$ ,  $\beta$ , and  $f$ , were set to those used in the experiments in Section 4.2. For all four data, the p-values of IUOIL, HUOMIL\*, and HUOMI are computed as 0.00000, and the mean values of DUOCA are always less than those of HUOMIL\* and HUOMI under the same conditions. Compared with the typically used significance level of 0.05 (5%), all p-values are less than 0.05, implying that the defined null

hypothesis is rejected. Accordingly, the proposed DUOCA demonstrates statistical significance in discovering a narrower and more refined set of high utility occupancy patterns through the damped window control.

#### 4.7. Case study with concept-drifting test

In this section, we analyze the results of the case study with real data presented in Table 4. The decaying factors are set as 0.999888, 0.999915, 0.999993, and 0.999992 in Chess, Mushroom, Pumsb, and



Retail, respectively. The minimum support threshold,  $\alpha$ , is fixed, while the minimum utility occupancy threshold,  $\beta$ , is varied for the data. The results demonstrate that the proposed DUOCA outperforms the comparison method, IUOIL, in a data control services environment where data streams accumulate continuously in real time. The case study consists of a concept drifting test based on algorithms, DUOCA and IUOIL, reflected in the forgetting concept [35] to demonstrate the effectiveness of the proposed method in real-world practicality. The forgetting concept was suggested to represent the organization of data updated in real-time. Accordingly, the concept drifting test introduced in the case study aims to compare the performance of DUOCA and IUOIL for stream environments in the real world by dealing with new data that reflects the latest trends along with the deletion of old data. The forgetting interval indicates a value for how much data each algorithm accumulates while discarding. For example, F16K means the algorithm discards the oldest 16,000 transactions in the data while scanning the same number of transactions to extract the results. Tables 9 and 10 show the runtime and memory usage for the two algorithms measured in the evaluation, presented as the forgetting intervals. For example, for the results of Pumsb, DUOCA requires 18.586 and 33.525 seconds with 124.7 and 125.3 MB in F4K and F2K, respectively. However, IUOIL requires 47.331 and 83.332 seconds with 179.1 and 180.6 MB in F4K and F2K, respectively. The concept drifting test results demonstrated that the proposed DUOCA performs more efficiently than the state-of-the-art method, IUOIL, in an environment where data deletion or insertion occurs in stream environments. Moreover, since the proposed method considers the time flow of each pattern in an environment where data is updated, it exhibits the difference that deals with results that reflect the recent trends in the latest data compared to the other method.

## 5. Conclusions

A novel approach for discovering damped high utility occupancy patterns derived from the damped window control was proposed in this paper. The extracted results of DUOCA were more refined than those of previous approaches because it considers the arrival time of transactions to consider their relative importance. The process of assigning relative importance depending on the time flow of scanned data enabled the extraction of recent insightful patterns from stream environments. In addition, new upper bound values and pruning strategies were introduced to increase the efficiency of the pattern expansion process. These strategies were designed to consider advanced damped window control systems, and the system contributed to diminishing the search space without any loss or duplication. Furthermore, the single scan-based construction and restructuring process of DUOCA for newly scanned transactions ensured effective processing of time-sensitive information in dynamic environments. Concerning performance analysis, the experimental results demonstrated the superiority for runtime and scalability with comparable memory usage in stream environments while handling the significance of results. The ablation study showed the effectiveness of the proposed strategies, and the significance test indicated that the proposed method analyzed statistically fewer result patterns. Moreover, a case study with the concept-drifting test demonstrated that the proposed method has an advantage in an environment where the handled data streams are updated in real-time. In our future work, we plan to discuss deriving a more efficient strategy by considering advanced data structures and new pruning strategies.

## CRedit authorship contribution statement

**Taewoong Ryu:** Writing – original draft, Validation, Methodology. **Deoyoung Kim:** Writing – original draft, Validation, Methodology. **Seungwan Park:** Writing – original draft, Validation. **Seongbin Park:** Writing – original draft, Validation. **Myungha Cho:** Validation, Methodology. **Hanju Kim:** Validation, Methodology. **Junyoung Park:** Validation, Methodology. **Hyeonmo Kim:** Validation, Methodology. **Unil**

**Yun:** Writing – original draft, Validation, Supervision, Methodology, Investigation, Formal analysis, Conceptualization.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

<http://fimi.ua.ac.be/data/> (Real dataset used in the performance evaluations can be acquired from the FIMI repository (.))

## References

- [1] A. Jazayeri, C.C. Yang, Frequent pattern mining in continuous-time temporal networks, *IEE Trans. Pattern. Anal. Mach. Intell.* 46 (1) (2024) 305–321.
- [2] R. Davashi, ITUPP: A fast method for interactive mining of Top-K frequent patterns from uncertain data, *Expert. Syst. Appl.* 214 (2023) 119156.
- [3] Y. Wu, Q. Hu, Y. Li, et al., OPP-miner: order-preserving sequential pattern mining for time series, *IEE Trans. Cybern.* 53 (5) (2023) 3288–3300.
- [4] Y. Wu, X. Zhao, Y. Li, et al., OPR-Miner: order-preserving rule mining for time series, *IEE Trans. Knowl. Data Eng.* 35 (11) (2023) 11722–11735.
- [5] H. Kim, M. Cho, T. Ryu, S. Park, D. Kim, D. Kim, H. Kim, H. Kim, U. Yun, DFPM: damped window-based flexible periodic pattern analysis on the time-decaying model, *Future Gener. Comput. Syst.* (2025) 107970.
- [6] H. Kim, H. Kim, M. Cho, T. Ryu, C. Lee, U. Yun, Regularity-driven pattern extraction and analysis approach by the pre-pruning technique without pattern loss, *Future Gener. Comput. Syst.* 166 (2025) 107670.
- [7] H. Kim, M. Cho, S. Park, D. Kim, D. Kim, T. Ryu, C. Lee, U. Yun, Temporal fuzzy utility-based data analysis on data streams, *Expert. Syst. Appl.* 284 (2025) 127836.
- [8] G. Lee, U. Yun, H. Ryang, Mining weighted erasable patterns by using underestimated constraint-based pruning technique, *J. Intell. Fuzzy Syst.* 28 (2015) 1145–1157.
- [9] G. Lee, U. Yun, H. Ryang, D. Kim, Erasable itemset mining over incremental databases with weight conditions, *Eng. Appl. Artif. Intell.* 52 (2016) 213–234.
- [10] J. Jia, G. Ji, B. Zhao, G. Ji, Mining converging patterns over streaming trajectories of moving objects in road networks, *Knowl. Based. Syst.* 309 (2025) 112883.
- [11] M. Iqbal, A.S.D. Prayitno, H.-K. Pao, I. Mukhlash, Mining fuzzy local periodic activity pattern for Smart home applications, *Knowl. Based. Syst.* 293 (2024) 111629.
- [12] H.V. Duong, T.C. Truong, B. Le, P. Fournier-Viger, Mining interesting sequential patterns using a novel balanced utility measure, *Knowl. Based. Syst.* 294 (2024) 111796.
- [13] T.D.D. Nguyen, N.T. Tung, T. Pham, L.T.T. Nguyen, Parallel approaches to extract multi-level high utility itemsets from hierarchical transaction databases, *Knowl. Based. Syst.* 276 (2023) 110733.
- [14] N.T. Tung, T.D.D. Nguyen, L.T.T. Nguyen, B. Vo, An efficient method for mining high-utility itemsets from unstable negative profit databases, *Expert. Syst. Appl.* 237 (2024) 121489.
- [15] B. Huynh, N.T. Tung, T.D.D. Nguyen, Q.-T. Bui, L.T.T. Nguyen, U. Yun, B. Vo, An efficient strategy for mining high-efficiency itemsets in quantitative databases, *Knowl. Based. Syst.* 299 (2024) 112035.
- [16] H.M. Huynh, B. Vo, Z.K. Oplatková, W. Pedrycz, An approach for incremental mining of clickstream patterns as a service application, *IEE Trans. Serv. Comput.* 16 (6) (2023) 3892–3905.
- [17] J. Kim, U. Yun, H. Kim, et al., Average utility driven data analytics on damped windows for intelligent systems with data streams, *Int. J. Intell. Syst.* 36 (10) (2021) 5741–5769.
- [18] L.T.T. Nguyen, D.-B. Vu, T.D.D. Nguyen, B. Vo, Mining maximal high utility itemsets on dynamic profit databases, *Cybern. Syst.* 51 (2) (2020) 140–160.
- [19] M. Han, N. Zhang, L. Wang, X. Li, H. Cheng, Mining high utility pattern with negative items in dynamic databases, *Int. J. Intell. Syst.* 37 (8) (2022) 5325–5353.
- [20] D. Kim, H. Kim, S. Park, H. Kim, M. Cho, S. Park, T. Ryu, C. Lee, H. Kim, U. Yun, Efficient mining of incremental high utility patterns with negative unit profits over all the accumulated stream data, *Knowl. Based. Syst.* (2025) 113956.
- [21] Z. Li, G. Li, L. Zhao, T. Shang, List-based mining top-k average-utility itemsets with effective pruning and threshold raising strategies, *Appl. Intell.* 53 (21) (2023) 25678–25696.
- [22] K. Huang, Y. Cui, Q. Ye, Y. Zhao, X. Zhao, Y. Tian, K. Zheng, H. Hu, X. Zhou, TED+: towards discovering top-k edge-diversified patterns in a graph database, *IEE Trans. Knowl. Data Eng.* 36 (5) (2024) 2224–2238.
- [23] T.D.D. Nguyen, L.T.T. Nguyen, L. Vu, et al., Efficient algorithms for mining closed high utility itemsets in dynamic profit databases, *Expert. Syst. Appl.* 186 (2021) 115741.
- [24] X. Zhao, X. Zhong, B. Han, Frequent closed high-utility itemset mining algorithm based on Leiden community detection and compact genetic algorithm, *IEE Access.* 12 (2024) 84763–84773.

- [25] G. Li, T. Shang, Y. Zhang, Efficient mining high average-utility itemsets with effective pruning strategies and novel list structure, *Appl. Intell.* 53 (5) (2023) 6099–6118.
- [26] H. Kim, H. Kim, M. Cho, et al., Efficient approach of high average utility pattern mining with indexed list-based structure in dynamic environments, *Inf. Sci.* 657 (2024) 119924.
- [27] Y. Baek, U. Yun, H. Kim, J. Kim, B. Vo, T.C. Truong, Z.-H. Deng, Approximate high utility itemset mining in noisy environments, *Knowl. Based. Syst.* 212 (2021) 106596.
- [28] H.-J. Choi, C.H. Park, Emerging topic detection in twitter stream based on high utility pattern mining, *Expert. Syst. Appl.* 115 (2019) 27–36.
- [29] K. K. Sethi, D. Ramesh, A. Rathore, and S. Sarin, "HUIM-SMP: high utility itemset mining based stock market analogy," in *Proc. 10th Int. Conf. Comput., Commun. Netw. Technol.* (2019) 1–6.
- [30] Liu, C. Guo, Mining top-N high-utility operation patterns for taxi drivers, *Expert. Syst. Appl.* 170 (2021) 114546.
- [31] L. Tang, L. Zhang, P. Luo, M. Wang, Incorporating occupancy into frequent pattern mining for high quality pattern recommendation, in: *Proc. 21st ACM Int. Conf. Inf. Knowl. Manag.*, 2012, pp. 75–84.
- [32] J. Adhikari, Occupancy based pattern mining: current status and future directions, *Int. J. Next Gener. Comput.* 11 (1) (2020).
- [33] G.M. Karthik, M. Sayeekumar, R. Kumaravel, T. Aravind, Finding spectrum occupancy pattern using CBFP mining technique, *J. Intell. Fuzzy Syst.* 39 (3) (2020) 4361–4368.
- [34] Z.Wen Shen, Y. Zhao, et al., OCEAN: fast discovery of high utility occupancy itemsets, in: *Proc. Pac.-Asia Conf. Knowl. Disc. Data Min.*, 2016, pp. 354–365.
- [35] M. Cho, H. Kim, Y. Baek, S. Park, D. Kim, D. Kim, C. Lee, B. Vo, W. Pedrycz, U. Yun, Uncertainty-driven pattern mining on incremental data for stream analyzing service, *IEEe Trans. Serv. Comput.* 18 (2) (2025) 1081–1096.
- [36] Y. Liu, Real-time multi-class detection of colorectal polyps based on the colon-YOLO network, *Alexandria Eng. J.* 127 (2025) 595–605.
- [37] Y.T. Wen, H.-K. Yang, W.-C. Peng, Mining willing-to-pay behavior patterns from payment datasets, *ACM. Trans. Intell. Syst. Technol.* 13 (1) (2022) 19, 141–14.
- [38] H. Kim, U. Yun, Y. Baek, et al., Damped sliding based utility oriented pattern mining over stream data, *Knowl. Based. Syst.* 213 (2021) 106653.
- [39] H. Nam, U. Yun, B. Vo, et al., Efficient approach for damped window-based high utility pattern mining with list structure, *IEEe Access.* 8 (2020) 50958–50968.
- [40] L. Zhang, P. Luo, L. Tang, et al., Occupancy-based frequent pattern mining, *ACM. Trans. Knowl. Discov. Data* 10 (2) (2015) 33, 141–14.
- [41] Z.-H. Deng, Mining high occupancy itemsets, *Future Gener. Comput. Syst.* 102 (2020) 222–229.
- [42] S. Data, K. Mali, U. Ghosh, High occupancy itemset mining with consideration of transaction occupancy, *Arab. J. Sci. Eng.* 47 (2022) 2061–2075.
- [43] H. Kim, T. Ryu, C. Lee, et al., Mining high occupancy patterns to analyze incremental data in intelligent systems, *ISA Trans.* 131 (2022) 460–475.
- [44] W. Gan, J.C.-W. Lin, P. Fournier-Viger, et al., HUOPM: high-utility occupancy pattern mining, *IEEe Trans. Cybern.* 50 (3) (2020) 1195–1208.
- [45] T. Ryu, U. Yun, C. Lee, et al., Occupancy-based utility pattern mining in dynamic environments of intelligent systems, *Int. J. Intell. Syst.* 37 (9) (2022) 5477–5507.
- [46] H. Kim, T. Ryu, C. Lee, et al., Efficient method for mining high utility occupancy patterns based on indexed list structure, *IEEe Access.* 11 (2023) 43140–43158.
- [47] M. Cho, H. Kim, S. Park, D. Kim, D. Kim, U. Yun, Advanced approach for mining utility occupancy patterns in incremental environment, *Knowl. Based. Syst.* 306 (2024) 112713.
- [48] H. Nam, U. Yun, E. Yoon, J.C.-W. Lin, Efficient approach of recent high utility stream pattern mining with indexed list structure and pruning strategy considering arrival times of transactions, *Inf. Sci.* 529 (2020) 1–27.
- [49] Y. Baek, U. Yun, H. Kim, et al., RHUPS: mining recent high utility patterns with sliding window-based arrival time control over data streams, *ACM. Trans. Intell. Syst. Technol.* 12 (2) (2021) 27, 161–16.
- [50] J.C.-W. Lin, M. Pirouz, Y. Djenouri, et al., Incrementally updating the high average-utility patterns with pre-large concept, *Appl. Intell.* 50 (11) (2020) 3788–3807.
- [51] H. Kim, M. Cho, H. Nam, et al., Advanced incremental erasable pattern mining from the time-sensitive data stream, *Knowl. Based. Syst.* 299 (2024) 112001.
- [52] S. Krishnamoorthy, HMiner: efficiently mining high utility itemsets, *Expert. Syst. Appl.* 90 (2017) 168–183.
- [53] B. Vo, L.V. Nguyen, V.V. Vu, et al., Mining correlated high utility itemsets in one phase, *IEEe Access.* 8 (2020) 90465–90477.